Math Formalism for EP/TPD Unification

Alexander S. Carney

Generated using MATLAB R2022b Symbolic Toolbox, Version 9.2

Table of Contents

State Space Model

Equations 1 - 4

```
clear
syms kappa_c kappa_y f_c f_y J phi f_d real positive
syms f_tilde_c f_tilde_y f_tilde_d kappa_tilde_bar real positive
syms kappa_tilde_c kappa_tilde_y real
syms Delta_tilde_f Delta_tilde_kappa Delta_f Delta_kappa real
A = [-kappa_c/2 - 1j * (f_c) -1j * J  ; -1j * J * exp(1j * phi) -kappa_y/2 - 1j * (f_y)]
```

```
A =
```

$$\begin{pmatrix} -\frac{\kappa_c}{2} - f_c\,\mathrm{i} & -J\,\mathrm{i} \\ -J\,\mathrm{e}^{\phi\,\mathrm{i}}\,\mathrm{i} & -\frac{\kappa_y}{2} - f_y\,\mathrm{i} \end{pmatrix}$$

We normalize by dividing by J. This allows us to work in non-dimensionalized units and will simplify the future math. We assume J != 0

```
A_tilde = A/J;
A_tilde = subs(A_tilde, [f_y, kappa_y/2], [f_c - Delta_f, kappa_c/2 - Delta_kappa]);
A_tilde = subs(expand(A_tilde), ...
     [kappa_c, f_c, f_d, Delta_kappa, Delta_f]/J, ...
     [kappa_tilde_c, f_tilde_c, f_tilde_d, Delta_tilde_kappa, Delta_tilde_f])
```

A_tilde =

$$\begin{pmatrix} -\frac{\tilde{\kappa}_c}{2} - \tilde{f}_c\,\mathrm{i} & -\mathrm{i} \\ -\mathrm{e}^{\phi\,\mathrm{i}}\,\mathrm{i} & \tilde{\Delta}_\kappa - \frac{\tilde{\kappa}_c}{2} + \tilde{\Delta}_f\,\mathrm{i} - \tilde{f}_c\,\mathrm{i} \end{pmatrix}$$

Equation 3

Now get steady state transmission

```
syms a real positive
B = [1; 0]; C = [0 1];
ssrepsonse = abs(C * inv(1j * f_tilde_d * eye(2) + A_tilde) * B)^2;
beta_ss = simplify(ssrepsonse, 'Steps', 50)
```

beta_ss =

$$\frac{16}{\left(-4\,\tilde{f}_c^{\,2} + 8\,\tilde{f}_c\,\tilde{f}_d + 4\,\tilde{\Delta}_f\,\tilde{f}_c - 4\,\tilde{f}_d^{\,2} - 4\,\tilde{\Delta}_f\,\tilde{f}_d + \tilde{\kappa}_c^{\,2} - 2\,\tilde{\Delta}_\kappa\,\tilde{\kappa}_c + 4\cos(\phi)\right)^2 + \left(4\sin(\phi) - 4\,\tilde{\Delta}_\kappa\,\tilde{f}_c + 4\right.}$$

The equation is so unwieldly, it doesn't even fit on the page. How are we going to analyze extrema of this equation in closed form?

Although β_{ss} is unweildy, we can see that it is written in the form:

$\beta_{\mathrm{ss}} = \frac{16}{A\left(\tilde{f}_d\right)^2 + B\left(\tilde{f}_d\right)^2}$, where A and B are placeholders for expressions that are quadratic (A) and linear (B) in $\tilde{f}_d$.

The extrema of β_{ss} are given by the condition $\frac{\partial}{\partial \tilde{f}_d}\beta_{\mathrm{ss}} = 0$, which we show below simplifies into a cubic expression with compact coefficients!

This is not the case for other drive/readout configurations (Appdx)

Eigenspectrum Analysis

Eigenvalue Splitting (Eqs. 5-6)

```
es = eig(A_tilde);
spec_split = es(2) - es(1)
```

spec_split =

$$\sqrt{-\widetilde{\Delta}_f^{\,2} + 2\,\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa\,\mathrm{i} + \widetilde{\Delta}_\kappa^{\,2} - 4\,\mathrm{e}^{\phi\,\mathrm{i}}}$$

```
% equation 5 provides same output
expand(sqrt( (Delta_tilde_kappa + 1j * Delta_tilde_f)^2 - 4 * exp(1j * phi)))
```

ans =

$$\sqrt{-\widetilde{\Delta}_f^{\,2} + 2\,\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa\,\mathrm{i} + \widetilde{\Delta}_\kappa^{\,2} - 4\,\mathrm{e}^{\phi\,\mathrm{i}}}$$

```
syms Delta_tilde_lambda
% save for later
ss = subs(spec_split, spec_split, Delta_tilde_lambda)
```

ss =

$$\widetilde{\Delta}_\lambda$$

```
lambda_pm = [subs(es(1), spec_split, Delta_tilde_lambda); subs(es(2), spec_split, Delta_tilde_l
```

lambda_pm =

$$\begin{pmatrix} \frac{\widetilde{\Delta}_\kappa}{2} - \frac{\widetilde{\Delta}_\lambda}{2} - \frac{\widetilde{\kappa}_c}{2} + \frac{\widetilde{\Delta}_f\,\mathrm{i}}{2} - \widetilde{f}_c\,\mathrm{i} \\ \frac{\widetilde{\Delta}_\kappa}{2} + \frac{\widetilde{\Delta}_\lambda}{2} - \frac{\widetilde{\kappa}_c}{2} + \frac{\widetilde{\Delta}_f\,\mathrm{i}}{2} - \widetilde{f}_c\,\mathrm{i} \end{pmatrix}$$

Petermann Factor (Eq. 7)

Next, we solve for the Petermann factor. In the Appendix, we explain the steps that we are doing here, and supply a useful reference.

```
% Now, can we find the Resolvent
syms z
Rz = inv( A_tilde - z*eye(2) );
% The residues of the Resolvent are the eigenprojectors
P1 = simplify(-series(Rz, z, es(1), 'Order', 2) * (z - es(1)));
P1 = subs(P1, spec_split, Delta_tilde_lambda);
P2 = simplify(-series(Rz, z, es(2), 'Order', 2) * (z - es(2)));
P2 = subs(P2, spec_split, Delta_tilde_lambda);
normP1_2 = simplify(sqrt(trace(P1' * P1)), 'Steps', 50);
normP2_2 = simplify(sqrt(trace(P2' * P2)), 'Steps', 50);
```

```
Petermann_Factor = 1/2 * ( normP1_2^2 + normP2_2^2 )
```

Petermann_Factor =

$$\frac{\widetilde{\Delta}_f^{\,2} + \widetilde{\Delta}_\kappa^{\,2} + \left|\widetilde{\Delta}_\lambda\right|^2 + 4}{2\left|\widetilde{\Delta}_\lambda\right|^2}$$

Manifold of EPs (Eq. 8)

To find EPs, we find where the Petermann factor diverges. We solve $\widetilde{\Delta}_\lambda \to 0$.

```
l = abs(spec_split);
assume(Delta_tilde_f, 'real')
s = solve(l == 0, [Delta_tilde_f Delta_tilde_kappa], 'ReturnConditions', true);
Delta_fs_are = simplify(s.Delta_tilde_f, 'Steps', 50)
```

Delta_fs_are =

$$\begin{pmatrix} 2\sin\left(\frac{\phi}{2}\right) - 4\cos\left(\frac{\phi}{2}\right)\mathrm{i} \\ -2\sin\left(\frac{\phi}{2}\right) \\ 2\sin\left(\frac{\phi}{2}\right) \end{pmatrix}$$

```
Delta_kappas_are = s.Delta_tilde_kappa
```

Delta_kappas_are =

$$\begin{pmatrix} -2\cos\left(\frac{\phi}{2}\right) \\ -2\cos\left(\frac{\phi}{2}\right) \\ 2\cos\left(\frac{\phi}{2}\right) \end{pmatrix}$$

```
s.conditions;
```

Here, we get the parameterization of the exceptional surface. The entry with complex $\widetilde{\Delta}_f$ is an extra artifact of the symbolic toolbox, not a legitimate solution within our domain.

Transmission Spectrum Analysis

Depressed Cubic Exact Form (Eqs. 9-11)

We see that the steady state transmission amplitudes $\widetilde{\beta}_{ss}$ attain a very complicated solution. **It is hard to believe at this point that a simple closed form solution for the peaks will exist...**

Continuing, we look for the critical points.

```
dss = diff(ssrepsonse, f_tilde_d) == 0;
s = solve(dss, f_tilde_d, 'ReturnConditions', true);
```

```
s_f_tilde_d = s.f_tilde_d;
% At this point, the solution are the 3 roots of a complicated cubic
sch = children(s_f_tilde_d(1));
pol = simplify(sch{1}, 'Steps', 50);
syms z
% Isolate the terms of the cubic for further analysis
pol_col = collect(pol, z)
```

pol_col =

$$(-8\,\mathrm{e}^{\phi\,\mathrm{i}})\,z^3+\left(-12\,\widetilde{\Delta}_f\,\mathrm{e}^{\phi\,\mathrm{i}}+24\,\widetilde{f}_c\,\mathrm{e}^{\phi\,\mathrm{i}}\right)z^2+\left(-4\,\mathrm{e}^{\phi\,\mathrm{i}}\,\widetilde{\Delta}_f^{\,2}+24\,\mathrm{e}^{\phi\,\mathrm{i}}\,\widetilde{\Delta}_f\,\widetilde{f}_c-4\,\mathrm{e}^{\phi\,\mathrm{i}}\,\widetilde{\Delta}_\kappa^{\,2}+4\,\mathrm{e}^{\phi\,\mathrm{i}}\,\widetilde{\Delta}_\kappa\,\widetilde{\kappa}_c-24\,\mathrm{e}^{q}\right.$$

Here, we see that the critical points are the roots of an extremely complicated cubic equation.

After some inspiration from a documentary about the first methods to solve cubic equations, we try a depressed cubic transformation. This transformation takes the cubic:

$$az^3+bz^2+cz+d=0$$

and applies the substitution $z=u-b/(3a)$ to force $a=1, b=0$.

Let's see if this substitution can simplify the equation

```
[coeff_list, powers] = coeffs(pol_col, z, 'All');
syms u
b = coeff_list(2);
a = coeff_list(1);
% depressed cubic sub
z_sub = u - b/(3 * a);
haileycubic = simplify(subs(pol_col, z, z_sub), 'Steps', 50);
haileycubic = simplify(rewrite(haileycubic, 'exp'), 'Steps', 50);
haileycubic = subs(haileycubic, cos(phi) + 1j * sin(phi), exp(1j * phi));
[hcoeff, hpow] = coeffs(haileycubic, u, 'All');
hnorm = simplify(haileycubic / hcoeff(1), 'Steps', 50);
[hco, hpo] = coeffs(hnorm, u, 'All')
```

hco =

$$\left(1\quad 0\quad -\frac{\widetilde{\Delta}_f^{\,2}}{4}+\frac{\widetilde{\Delta}_\kappa^{\,2}}{2}-\frac{\widetilde{\Delta}_\kappa\,\widetilde{\kappa}_c}{2}+\frac{\widetilde{\kappa}_c^{\,2}}{4}-\cos(\phi)\quad \frac{\widetilde{\Delta}_\kappa\sin(\phi)}{2}-\frac{\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa^{\,2}}{4}-\frac{\widetilde{\kappa}_c\sin(\phi)}{2}+\frac{\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa\,\widetilde{\kappa}_c}{4}\right)$$

hpo = $\left(u^3\quad u^2\quad u\quad 1\right)$

```
syms kappa_bar real positive
```

Now, we have a cubic of the form:

$f^3+pf+q=0$, let's see if p and q are simpler.

```
porig = hco(3)
```

porig =

$$-\frac{\widetilde{\Delta}_f^{\,2}}{4}+\frac{\widetilde{\Delta}_\kappa^{\,2}}{2}-\frac{\widetilde{\Delta}_\kappa\,\widetilde{\kappa}_c}{2}+\frac{\widetilde{\kappa}_c^{\,2}}{4}-\cos(\phi)$$

```
qorig = hco(4)
```

qorig =

$$\frac{\widetilde{\Delta}_\kappa\sin(\phi)}{2}-\frac{\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa^{\,2}}{4}-\frac{\widetilde{\kappa}_c\sin(\phi)}{2}+\frac{\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa\,\widetilde{\kappa}_c}{4}$$

Raw original form of q and p is above.

```
kt_bar = kappa_tilde_bar == (kappa_tilde_c + kappa_tilde_y)/2;
dk = Delta_tilde_kappa == (kappa_tilde_c-kappa_tilde_y)/2;
ktc_is = solve(dk, kappa_tilde_c);
kty_is = solve(kt_bar, kappa_tilde_y);
simplify(kappa_tilde_c == subs(ktc_is, kappa_tilde_y, kty_is))
```

ans =

$$\widetilde{\Delta}_\kappa+\overline{\widetilde{\kappa}}=\widetilde{\kappa}_c$$

Use this substitution to simplify the equations

```
p_raw = simplify(subs(porig, kappa_tilde_c, kappa_tilde_bar + Delta_tilde_kappa))
```

p_raw =

$$-\frac{\widetilde{\Delta}_f^{\,2}}{4}+\frac{\widetilde{\Delta}_\kappa^{\,2}}{4}+\frac{\overline{\widetilde{\kappa}}^{2}}{4}-\cos(\phi)$$

```
q_raw = simplify(subs(qorig, kappa_tilde_c, kappa_tilde_bar + Delta_tilde_kappa))
```

q_raw =

$$-\frac{\overline{\widetilde{\kappa}}\left(2\sin(\phi)-\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa\right)}{4}$$

Interestingly, it looks like these cubic coefficients have a relationship to the eigenvalue splitting from above. Can we write p and q in terms of the eigenvalue splitting?

```
ps = simplify(subs(p_raw, real(spec_split^2)/4, real(Delta_tilde_lambda^2)/4))
```

ps =

$$\frac{\overline{\widetilde{\kappa}}^{2}}{4}+\frac{\mathrm{real}\left(\widetilde{\Delta}_\lambda^{\,2}\right)}{4}$$

```
qs = simplify(subs(q_raw, imag(spec_split^2)/2, imag(Delta_tilde_lambda^2)/2))
```

qs =

$$\frac{\overline{\widetilde{\kappa}}\,\mathrm{imag}\left(\widetilde{\Delta}_\lambda^{\,2}\right)}{8}$$

```
p_tilde_physical = simplify(subs(ps, kappa_tilde_bar, kappa_tilde_c - Delta_tilde_kappa), 'Step
```

p_tilde_physical =

$$\frac{\mathrm{real}\left(\widetilde{\Delta}_\lambda^{\,2}\right)}{4}+\frac{\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)^2}{4}$$

```
q_tilde_physical = simplify(subs(qs, kappa_tilde_bar, kappa_tilde_c - Delta_tilde_kappa), 'Step
```

q_tilde_physical =

$$-\frac{\mathrm{imag}\left(\widetilde{\Delta}_\lambda^{\,2}\right)\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)}{8}$$

```
p_tilde_is = subs(p_tilde_physical, Delta_tilde_lambda, spec_split);
q_tilde_is = subs(q_tilde_physical, Delta_tilde_lambda, spec_split);
```

It is extremely surprising to us that this simplified expression was hiding in the parameterization of the peak locations the whole time!

```
syms p_tilde q_tilde real
```

Cubic Discriminant (Eq. 12)

```
disc = -4 * p_tilde_is^3 - 27 * q_tilde_is^2
```

disc =

$$4\left(\cos(\phi)+\frac{\widetilde{\Delta}_f^{\,2}}{4}-\frac{\widetilde{\Delta}_\kappa^{\,2}}{4}-\frac{\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)^2}{4}\right)^3-\frac{27\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)^2\left(4\sin(\phi)-2\,\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa\right)^2}{64}$$

```
disc_pq = -4 * p_tilde^3 - 27 * q_tilde^2
```

disc_pq = $-4\,\widetilde{p}^3-27\,\widetilde{q}^2$

Parameterization of Transmission Peak Degeneracies

Here, we provide the full solution to the locations of all TPDs.

To start, we solve for the roots of the cubic equation. This can either be done numerically, or using Cardano's method, or using Viete's method (trig solution)

We choose Viete's method here, because the CAS can handle trig functions better than square and cube roots.

Vieta Trigonometric Solution (Eqs. 13-16)

```
syms theta theta_s
% Apply the Vieta trig form for the roots of depressed cubic
theta_val = (1/3) * acos(((3*q_tilde)/(2*p_tilde)) * sqrt(-3/p_tilde))
```

theta_val =

$$\frac{\operatorname{acos}\left(\frac{3\ \sqrt{3}\ \tilde{q}\ \sqrt{-\frac{1}{\tilde{p}}}}{2\,\tilde{p}}\right)}{3}$$

```
t0 = 2 * sqrt(-p_tilde/3) * cos( theta_val - (2*pi*0/3) );
t1 = 2 * sqrt(-p_tilde/3) * cos( theta_val - (2*pi*1/3) );
t2 = 2 * sqrt(-p_tilde/3) * cos( theta_val - (2*pi*2/3) );
nu_plus = subs(t0, theta_val, theta)
```

nu_plus =

$$2\cos(\theta)\ \sqrt{-\frac{\tilde{p}}{3}}$$

```
eta_root = subs(t1, theta_val, theta)
```

eta_root =

$$2\cos\left(\theta-\frac{2\,\pi}{3}\right)\ \sqrt{-\frac{\tilde{p}}{3}}$$

```
nu_minus = subs(t2, theta_val, theta)
```

nu_minus =

$$2\cos\left(\theta-\frac{4\,\pi}{3}\right)\ \sqrt{-\frac{\tilde{p}}{3}}$$

```

% Verification: Check that all of Vieta's relations apply
check_sum = simplify(eta_root + nu_plus + nu_minus)
```

check_sum = 0

```
% 2. Linear Coeff Check (Should be p_tilde)
check_p = simplify((eta_root * nu_plus) + (eta_root * nu_minus) + (nu_plus * nu_minus))
```

check_p = $\tilde{p}$

```
% 3. Constant Term Check (Should be -q_tilde)
check_q = simplify(t0 * t1 * t2, 'Steps', 50)
```

check_q =

$$\begin{cases} \widetilde{q} & \text{if } 0 < \widetilde{p} \\ -\widetilde{q} & \text{if } \widetilde{p} \le 0 \end{cases}$$

This check_q is particularly interesting, telling us that $\widetilde{p} \le 0$ in the split-peak regime.

For the single-peak regime, the single real root can be found using a complicated formula involving cosh and sinh.

We can use either the frequencies (which have an offset applied) or the roots themselvles for the splitting.

```
splitting_is = simplify(abs(nu_plus - nu_minus), 'All', true);
splitting_is = simplify(splitting_is(1))
```

```
splitting_is =
```

$$2\left|\sin\left(\theta+\frac{\pi}{3}\right)\right|\sqrt{|\widetilde{p}|}$$

```
full_splitting_eqn = subs(splitting_is, theta, theta_val);
full_splitting_eqn = simplify(subs(full_splitting_eqn, [p_tilde, q_tilde], [p_tilde_is, q_tilde
```

```
full_splitting_eqn =
```

$$2\left|\sin\left(\frac{\pi}{3}+\frac{\operatorname{acos}\left(3\sqrt{3}\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)\left(2\sin(\phi)-\widetilde{\Delta}_f\widetilde{\Delta}_\kappa\right)\left(\frac{1}{\widetilde{\Delta}_f^{\,2}-2\widetilde{\Delta}_\kappa^{\,2}+2\widetilde{\Delta}_\kappa\widetilde{\kappa}_c-\widetilde{\kappa}_c^{\,2}+4\cos(\phi)}\right)^{3/2}\right)}{3}\right)\right|.$$

Full TPD Parameterization for 2D TPDs (Appendix B - Eqs. 37-38)

TPDs occur when $\widetilde{p}=\widetilde{q}=0$

This is an unwieldly result! But it has all locations of TPDs within it.

Each "condition" matches TPD $\widetilde{\Delta}_\kappa, \widetilde{\Delta}_f$ locations to a condition based on $\phi, \widetilde{\kappa}_c$, the hyperparameters.

```
full_tpd_para = solve([q_tilde_is == 0, p_tilde_is == 0], [Delta_tilde_f, Delta_tilde_kappa],
```

```
full_tpd_para = struct with fields:
        Delta_tilde_f: [7×1 sym]
    Delta_tilde_kappa: [7×1 sym]
           parameters: x
           conditions: [7×1 sym]
```

```
delta_kappa_tpd = simplify(full_tpd_para.Delta_tilde_kappa, 'Steps', 50)
```

```
delta_kappa_tpd =
```

$$\begin{pmatrix} 0 \\ \widetilde{\kappa}_c \\ \widetilde{\kappa}_c \\ x \\ \frac{\widetilde{\kappa}_c}{2} - \frac{\sqrt{8\cos(\phi) - \widetilde{\kappa}_c^{\,2}}}{2} \\ \frac{\widetilde{\kappa}_c}{2} + \frac{\sqrt{8\cos(\phi) - \widetilde{\kappa}_c^{\,2}}}{2} \\ 0 \end{pmatrix}$$

```
delta_f_tpd = simplify(full_tpd_para.Delta_tilde_f, 'Steps', 50)
```

delta_f_tpd =

$$\begin{pmatrix} -\sqrt{\widetilde{\kappa}_c^{\,2} - 4\cos(\phi)} \\ \sqrt{\widetilde{\kappa}_c^{\,2} - 4\cos(\phi)} \\ -\sqrt{\widetilde{\kappa}_c^{\,2} - 4\cos(\phi)} \\ -\frac{x\left(-\widetilde{\kappa}_c^{\,2} + 2\,\widetilde{\kappa}_c\,x - 2\,x^2 + 4\cos(\phi)\right)}{2\sin(\phi)} \\ 0 \\ 0 \\ \sqrt{\widetilde{\kappa}_c^{\,2} - 4\cos(\phi)} \end{pmatrix}$$

```
tpd_conditions = simplify(full_tpd_para.conditions, 'Steps', 50)
```

tpd_conditions =

$$\begin{pmatrix} 4\cos(\phi) \le \widetilde{\kappa}_c^{\,2} \wedge \frac{\phi}{\pi} \in \mathbb{Z} \\ 4\cos(\phi) \le \widetilde{\kappa}_c^{\,2} \\ 4\cos(\phi) \le \widetilde{\kappa}_c^{\,2} \\ x^4 + 2\cos(\phi)^2 = \widetilde{\kappa}_c\,x^3 + \left(2\cos(\phi) - \frac{\widetilde{\kappa}_c^{\,2}}{2}\right)x^2 + 2 \wedge \frac{\phi}{\pi} \notin \mathbb{Z} \\ \widetilde{\kappa}_c^{\,2} \le 8\cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \\ \widetilde{\kappa}_c^{\,2} \le 8\cos(\phi) \wedge \frac{\phi}{\pi} \in \mathbb{Z} \\ 4\cos(\phi) \le \widetilde{\kappa}_c^{\,2} \wedge \frac{\phi}{\pi} \in \mathbb{Z} \end{pmatrix}$$

These equations encode the entire 2D TPD surface. Let's break it down similar to how we do in the Main Text:

Phase = 0 TPD Parameterization

```
tpd_para_phi_0 = solve(subs([q_tilde_is == 0, disc == 0], phi, 0), ...
                        [Delta_tilde_kappa, Delta_tilde_f], ...
                        'ReturnConditions', true);
dk_tpd_phi_0 = tpd_para_phi_0.Delta_tilde_kappa
```

dk_tpd_phi_0 =

$$\begin{pmatrix} \widetilde{\kappa}_c \\ 0 \\ \frac{\widetilde{\kappa}_c}{2} - \frac{\sqrt{8 - \widetilde{\kappa}_c^{\,2}}}{2} \\ \frac{\widetilde{\kappa}_c}{2} + \frac{\sqrt{8 - \widetilde{\kappa}_c^{\,2}}}{2} \\ 0 \\ \widetilde{\kappa}_c \end{pmatrix}$$

```
df_tpd_phi_0 = simplify(tpd_para_phi_0.Delta_tilde_f, 'Steps', 50)
```

df_tpd_phi_0 =

$$\begin{pmatrix} -\sqrt{\widetilde{\kappa}_c^{\,2} - 4} \\ \sqrt{\widetilde{\kappa}_c^{\,2} - 4} \\ 0 \\ 0 \\ -\sqrt{\widetilde{\kappa}_c^{\,2} - 4} \\ \sqrt{\widetilde{\kappa}_c^{\,2} - 4} \end{pmatrix}$$

```
simplify(tpd_para_phi_0.conditions, 'Steps', 50)
```

ans =

$$\begin{pmatrix} 0 \le \widetilde{\kappa}_c^{\,2} - 4 \\ 0 \le \widetilde{\kappa}_c^{\,2} - 4 \\ \widetilde{\kappa}_c^{\,2} - 8 \le 0 \\ \widetilde{\kappa}_c^{\,2} - 8 \le 0 \\ 0 \le \widetilde{\kappa}_c^{\,2} - 4 \\ 0 \le \widetilde{\kappa}_c^{\,2} - 4 \end{pmatrix}$$

For $\phi = 0$, there can either be two, four, or six TPDs, based on $\widetilde{\kappa}_c$.

The "primary" $\phi = 0$ TPD that we experimentally study in the main text is:

```
primary_tpd_phi_0 = [dk_tpd_phi_0(3) ; df_tpd_phi_0(3)]
```

primary_tpd_phi_0 =

$$\begin{pmatrix} \frac{\widetilde{\kappa}_c}{2} - \frac{\sqrt{8 - \widetilde{\kappa}_c^{\,2}}}{2} \\ 0 \end{pmatrix}$$

Phase = pi TPD Parameterization

```
tpd_para_phi_pi = solve(subs([q_tilde_is == 0, p_tilde_is == 0], phi, pi), ...
                        [Delta_tilde_kappa, Delta_tilde_f], ...
                        'ReturnConditions', true);
dk_tpd_phi_pi = tpd_para_phi_pi.Delta_tilde_kappa
```

dk_tpd_phi_pi =

$$\begin{pmatrix} \widetilde{\kappa}_c \\ \widetilde{\kappa}_c \\ 0 \\ 0 \end{pmatrix}$$

```
df_tpd_phi_pi = tpd_para_phi_pi.Delta_tilde_f
```

df_tpd_phi_pi =

$$\begin{pmatrix} \sqrt{\widetilde{\kappa}_c^{\,2} + 4} \\ -\sqrt{\widetilde{\kappa}_c^{\,2} + 4} \\ \sqrt{\widetilde{\kappa}_c^{\,2} + 4} \\ -\sqrt{\widetilde{\kappa}_c^{\,2} + 4} \end{pmatrix}$$

```
tpd_para_phi_pi.conditions
```

ans =

$$\begin{pmatrix} \text{symtrue} \\ \text{symtrue} \\ \text{symtrue} \\ \text{symtrue} \end{pmatrix}$$

This case is much simpler. There are always four TPDs.

The "primary" $\phi = \pi$ TPD that we experimentally study in the main text is:

```
primary_tpd_phi_pi = [dk_tpd_phi_pi(3) ; df_tpd_phi_pi(3)]
```

primary_tpd_phi_pi =

$$\begin{pmatrix} 0 \\ \sqrt{\widetilde{\kappa}_c^{\,2} + 4} \end{pmatrix}$$

Phase = pi/2 TPD Parameterization

```
tpd_para_phi_pi2 = solve(subs([q_tilde_is == 0, p_tilde_is == 0], phi, pi/2), ...
                         [Delta_tilde_kappa, Delta_tilde_f], ...
                         'ReturnConditions', true);
dk_tpd_phi_pi2 = tpd_para_phi_pi2.Delta_tilde_kappa
```

dk_tpd_phi_pi2 =

$$\begin{pmatrix} -\frac{\widetilde{\kappa}_c^{\,2}\, x}{4} + \widetilde{\kappa}_c + \frac{x^3}{4} \\ \widetilde{\kappa}_c \\ \widetilde{\kappa}_c \end{pmatrix}$$

```
df_tpd_phi_pi2 = tpd_para_phi_pi2.Delta_tilde_f
```

df_tpd_phi_pi2 =

$$\begin{pmatrix} x \\ -\widetilde{\kappa}_c \\ \widetilde{\kappa}_c \end{pmatrix}$$

```
tpd_phi_pi2_conditions = simplify(tpd_para_phi_pi2.conditions, 'Steps', 50)
```

tpd_phi_pi2_conditions =

$$\begin{pmatrix} \widetilde{\kappa}_c^{\,2}\, x^2 + 8 = x^4 + 4\,\widetilde{\kappa}_c\, x \\ \text{symtrue} \\ \text{symtrue} \end{pmatrix}$$

This quartic equation encodes the location of the TPDs. But, although it's a depressed quartic, it is not bi-quadratic. Therefore, the only way to get a closed form solution is with Ferrari's method.

```
xval = simplify(roots([1 0 -kappa_tilde_c^2 4 * kappa_tilde_c -8]));
```

Technically, a closed form solution for the TPD locations does exist. (xval is a closed-form solution)

However, the solution cannot even be printed on a page. It is better to handle this process numerically.

To solve for the TPD locations numerically, choose a value of $\widetilde{\kappa}_c$

```
xval_ktc_2 = double(vpa(subs(xval, kappa_tilde_c, 2)));
% drop the complex roots
```

```
xval_ktc_2 = xval_ktc_2(imag(xval_ktc_2) == 0)
```

```
xval_ktc_2 = 2×1
   -2.8051
    1.4934
```

Now, the roots (which are "x"), split into 2 real and 2 complex roots. Throw out the complex roots.

"Primary" $\phi = \frac{\pi}{2}$ TPD for $\widetilde{\kappa_c} = 2$

```
primary_tpd_phi_pi_ktc_2 = double([subs(dk_tpd_phi_pi2(1), [tpd_para_phi_pi2.parameters, kappa_
subs(df_tpd_phi_pi2(1), [tpd_para_phi_pi2.parameters, kappa_tilde_c], [xval_ktc_2(1), 2])    ])
```

```
primary_tpd_phi_pi_ktc_2 = 1×2
   -0.7130   -2.8051
```

This means that a closed form solution cannot be found for the subsequent figures of merit for intermediate phases.

TPD and EP Figures of Merit

EP Eigenvalue Splitting Strength (Appendix C.A)

First, what is the EP eigenvalue splitting strength

1) $\phi = 0$

```
syms epsilon_tilde real positive
split_rp = subs(imag(spec_split), [Delta_tilde_f, phi], [0 0])
```

```
split_rp =
```

$$\mathrm{imag}\left(\sqrt{\widetilde{\Delta}_{\kappa}^{\,2} - 4}\right)$$

```
split_rpep_ep = simplify(subs(split_rp, Delta_tilde_kappa, 2 - epsilon_tilde), 'Steps', 50)
```

```
split_rpep_ep =
```

$$\mathrm{imag}\left(\sqrt{\left(\widetilde{\varepsilon} - 2\right)^{2} - 4}\right)$$

```
series(split_rpep_ep, epsilon_tilde, 0, 'Order', 1)
```

```
ans =
```

$$2\sqrt{\widetilde{\varepsilon}}$$

2) $\phi = \pi$

```
split_arp = subs(imag(spec_split), [Delta_tilde_kappa, phi], [0 pi])
```

split_arp =

$$\mathrm{imag}\left(\sqrt{4-\widetilde{\Delta}_f^{\,2}}\right)$$

```
split_arpep_ep = simplify(subs(split_arp, Delta_tilde_f, 2 + epsilon_tilde), 'Steps', 50)
```

split_arpep_ep =

$$\sqrt{(\widetilde{\varepsilon}+2)^2-4}$$

```
series(split_arpep_ep, epsilon_tilde, 0, 'Order', 1)
```

ans =

$$2\sqrt{\widetilde{\varepsilon}}$$

3) $\phi = \pi/2$.

```
syms epsilon_tilde_h real positive
Delta_tilde_kappa_par = -sqrt(sym(2)) + epsilon_tilde_h;
Delta_tilde_f_par     = 2 / Delta_tilde_kappa_par;   % keeps the product 2
split_hyp = subs(imag(spec_split), ...
                 [Delta_tilde_kappa, Delta_tilde_f, phi], ...
                 [Delta_tilde_kappa_par, Delta_tilde_f_par, pi/2]);   % or any phi you need
split_hyp = simplify(split_hyp, 'Steps', 50)      % clean it up
```

split_hyp =

$$\mathrm{imag}\left(\sqrt{(\widetilde{\varepsilon}_h-\sqrt{2})^2-\frac{4}{(\widetilde{\varepsilon}_h-\sqrt{2})^2}}\right)$$

```
series(split_hyp, epsilon_tilde_h, 0, 'Order', 2)
```

ans =

$$2\,2^{1/4}\sqrt{\widetilde{\varepsilon}_h}$$

Here, we see that if you project the strength **onto a single axis**, the splitting strength is stronger by a factor of 2^(1/4).

However, we correct for this by analyzing the strength along the hyperbolic $\widetilde{q}=0$ path instead.

```
% ------------------------------------------------------------
% CORRECTED UNIT STEP - EPSILON IS DIRECTLY COMPARABLE NOW.
% 3) NR-EP  (phi = pi/2, hyperbolic path)
% ------------------------------------------------------------
% EP coordinates on the hyperbola  y = 2/x
x0 = -sqrt(sym(2));             % Delta_tilde_kappa at the EP
y0 = 2/x0;                      % Delta_tilde_f  at the EP ( = -sqrt(2) )
```

```
% Unit-tangent normalization  |(1,dy/dx)| = sqrt(1 + (2/x0^2)^2 )
norm_fac = sqrt( 1 + 4/x0^4 );  % equals sqrt(2) at this EP

% Convert arc-length step ε̃ to the x-coordinate increment
dx = epsilon_tilde_h / norm_fac;

Delta_tilde_kappa_par = x0 + dx;        % moves exactly ε̃ along the curve
Delta_tilde_f_par     = 2 / Delta_tilde_kappa_par;  % stays on the hyperbola

split_hyp = subs(imag(spec_split), ...
    [Delta_tilde_kappa, Delta_tilde_f, phi], ...
    [Delta_tilde_kappa_par, Delta_tilde_f_par, pi/2]);

split_hyp = simplify(split_hyp, 'Steps', 50);
series(split_hyp, epsilon_tilde_h, 0, 'Order', 2)
```

```
ans =
```

$$2\sqrt{\tilde{\varepsilon}_h}$$

Thus, for a perturbation that moves hyperbolically, the strength is equivalent to $\phi = 0, \pi$. For a perturbation that is either in $\tilde{\Delta}_\kappa$ or $\tilde{\Delta}_f$ ONLY, the strength is stronger.

TPD Peak Splitting Strength (Eqs. 18-19)

Here, we analyze the Puiseux series by using the `series` command.

1) $\phi = 0$

```
% Assume that the TPD exists, assume small perturbation.
assume(epsilon_tilde < sqrt(8 - kappa_tilde_c^2));
split_tpd_phi_0 = simplify(subs(full_splitting_eqn, [Delta_tilde_f, phi, Delta_tilde_kappa], [0
```

```
split_tpd_phi_0 =
```

$$\sqrt{2}\,\sqrt{|\tilde{\varepsilon}|}\,\sqrt{\sqrt{8-\tilde{\kappa}_c^{\,2}}-\tilde{\varepsilon}}$$

```
split_tpd_phi_0_series = (series(split_tpd_phi_0, epsilon_tilde, 0, 'Order', 1))
```

```
split_tpd_phi_0_series =
```

$$\sqrt{2}\,\left(8-\tilde{\kappa}_c^{\,2}\right)^{1/4}\sqrt{\frac{\tilde{\varepsilon}}{\operatorname{sign}(\tilde{\varepsilon})}}$$

```
assume(epsilon_tilde, 'clear')
```

the `sign` here implies $\tilde{\epsilon} > 0$

2) $\phi = \pi$

```
syms kappa_tilde_c real
split_tpd_phi_pi = simplify(subs(full_splitting_eqn, [Delta_tilde_f, phi, Delta_tilde_kappa], [
```

split_tpd_phi_pi =

$$\sqrt{\left|\widetilde{\varepsilon}+2\sqrt{\widetilde{\kappa}_c^{\,2}+4}\right|}\ \sqrt{\left|\widetilde{\varepsilon}\right|}$$

```
split_tpd_phi_pi_series = simplify(series(split_tpd_phi_pi, epsilon_tilde, 0, 'Order', 1))
```

split_tpd_phi_pi_series =

$$\sqrt{2}\left(\widetilde{\kappa}_c^{\,2}+4\right)^{1/4}\sqrt{\frac{1}{\operatorname{sign}\left(\widetilde{\varepsilon}+2\sqrt{\widetilde{\kappa}_c^{\,2}+4}\right)}}\sqrt{\frac{\widetilde{\varepsilon}}{\operatorname{sign}\left(\widetilde{\varepsilon}\right)}}$$

the sign here implies $\widetilde{\epsilon}>0$

3) $\phi=\pi/2$

We can't find the strength in closed form, because we don't have an expression for the location of the TPD. We have to do this numerically by choosing $\widetilde{\kappa}_c$. This is done in python code instead.

Petermann Factor at TPD (Eqs. 20-21)

We know the Petermann Factor diverges at all EPs. But, what about TPDs?

```
PF_full = subs(Petermann_Factor, Delta_tilde_lambda, spec_split);
```

1) $\phi=0$

```
assumeAlso(kappa_tilde_c >= 0 & kappa_tilde_c < 2*sqrt(2));
simplify(subs(PF_full, [Delta_tilde_f, phi, Delta_tilde_kappa], [0, 0, primary_tpd_phi_0(1)]),
```

ans =

$$\left(-\frac{\dfrac{8}{\widetilde{\kappa}_c\sqrt{8-\widetilde{\kappa}_c^{\,2}}+4}}{\left(\dfrac{\widetilde{\kappa}_c}{2}-\dfrac{\sqrt{8-\widetilde{\kappa}_c^{\,2}}}{2}\right)^2-4}\right)$$

```
assume(kappa_tilde_c, 'clear')
```

2) $\phi=\pi$

```
assumeAlso(kappa_tilde_c >= 0)
simplify(subs(PF_full, [Delta_tilde_f, phi, Delta_tilde_kappa], [primary_tpd_phi_pi(2), pi, 0])
```

ans =

$$\frac{4}{\widetilde{\kappa}_c^{\,2}} + 1$$

Thermal Noise Efficiency (Eqs. 22-24)

We use the semiclassical Langevin/input-output convention in which uncorrelated themral noise from each mode reservoir j enters the mode equations with amplitude $\sqrt{\kappa_j}$.

This convention counts the resonantly transduced noise and *omits the*

direct input-output background term from the readout bath.

For an arbitrary readout mode r, under these assumptions:

$N_{\mathrm{th},r} = \sum_j \widetilde{\kappa}_r \widetilde{\kappa}_j |\widetilde{G}_{\mathrm{rj}}|^2$, where G is the Green's function

```
green = inv(1j * f_tilde_d * eye(2) + A_tilde)
```

green =

$$\begin{pmatrix} -\dfrac{2\left(2\,\widetilde{\Delta}_f - 2\,\widetilde{\Delta}_\kappa\,\mathrm{i} - 2\,\widetilde{f}_c + 2\,\widetilde{f}_d + \widetilde{\kappa}_c\,\mathrm{i}\right)}{\sigma_1} & -\dfrac{4}{\sigma_1} \\ -\dfrac{4\,\mathrm{e}^{\phi\,\mathrm{i}}}{\sigma_1} & -\dfrac{2\left(2\,\widetilde{f}_d - 2\,\widetilde{f}_c + \widetilde{\kappa}_c\,\mathrm{i}\right)}{\sigma_1} \end{pmatrix}$$

where

$$\sigma_1 = 4\,\mathrm{e}^{\phi\,\mathrm{i}}\,\mathrm{i} + 4\,\widetilde{\Delta}_f\,\widetilde{f}_c\,\mathrm{i} - 4\,\widetilde{\Delta}_f\,\widetilde{f}_d\,\mathrm{i} + 4\,\widetilde{\Delta}_\kappa\,\widetilde{f}_c - 4\,\widetilde{\Delta}_\kappa\,\widetilde{f}_d + 2\,\widetilde{\Delta}_f\,\widetilde{\kappa}_c - 2\,\widetilde{\Delta}_\kappa\,\widetilde{\kappa}_c\,\mathrm{i} + 8\,\widetilde{f}_c\,\widetilde{f}_d\,\mathrm{i} - 4\,\widetilde{f}_c\,\widetilde{\kappa}_c + 4$$

```
N_th = kappa_tilde_y * kappa_tilde_c * (green(2,1) * conj(green(2,1))) + kappa_tilde_y^2 * (gre
```

N_{th} is frequency dependent because each Green-function element is evaluated at the measurement drive frequency. For TPD figures of merit, we evaluate N_{th} at the degenerate transmission extremum.

In the shifted coordinate

$x = \widetilde{f}_d - \widetilde{f}_c + \widetilde{\Delta}_f/2$

The TPD satisfies $p = q = 0$, so all extrema coalesce at $x = 0$. Therefore,

$$f_d^{\mathrm{TPD}} = \widetilde{f}_c - \frac{\widetilde{\Delta}_f}{2}$$

We also replace $\widetilde{\kappa}_y$ with $\widetilde{\kappa}_c - 2\widetilde{\Delta}_\kappa$ to remove a variable

```
N_th = simplify(subs(N_th, [kappa_tilde_y, f_tilde_d], [kappa_tilde_c - 2 * Delta_tilde_kappa,
```

FIrst, phi = 0: (primary TPD)

It is important that we choose the primary TPD here because the TNE is NOT symmetric between the two

```
N_th_0 = subs(N_th, [Delta_tilde_kappa, Delta_tilde_f], [delta_kappa_tpd(5), 0]);
NTH0 = simplify(subs(N_th_0, phi, 0), 'Steps', 50)
```

NTH0 =

$$\frac{16\,\widetilde{\kappa}_c\,\sqrt{8-\widetilde{\kappa}_c^{\,2}}-64}{\left(\widetilde{\kappa}_c^{\,2}-4\right)^2}+4$$

Here we see a limitation of CAS systems. Matlab is writing this as if there is a singularity at $\widetilde{\kappa}_c = 2$, when there is none:

```
NTH0_primary_TPD = limit(NTH0, kappa_tilde_c, 2)
```

NTH0_primary_TPD = 2

The limit exists, this is a removable singularity.

The cleaned up expression is reported in the paper as:

$$N_{\mathrm{th}}(\phi=0)^{\mathrm{Primary}} = 4 - \frac{16}{\widetilde{\kappa}_c\sqrt{8-\widetilde{\kappa}_c^{\,2}}+4}$$

Interestingly, the singularity at $\widetilde{\kappa}_c = -2$ is **not** removable. For the "other" $\phi = 0$ TPD, $\widetilde{\kappa}_c = 2$ is a real singularity

$\widetilde{\kappa}_c$

Second, phi = pi (primary TPD)

Here, the choice of primary/other TPD does not matter because it is symmetric between the two

```
N_th_pi = subs(N_th, [Delta_tilde_kappa, Delta_tilde_f], [0, delta_f_tpd(2)]);
all_ans = simplify(subs(N_th_pi, phi, pi), 'All', true);
% This is the actual simple one in the paper
NTHpi_primary_TPD = all_ans(1)
```

NTHpi_primary_TPD =

$$\frac{2\,\widetilde{\kappa}_c^{\,2}+8}{\widetilde{\kappa}_c^{\,2}}$$

Single Root Closed Form Solution (Appendix C.D) (Eq. 40)

Here, we directly analyze the closed form solution for the Disc < 0 single-peak location.

```
syms delta_tilde real
t1_pre_split_p_larger_than_0 = simplify(-2 * sqrt(p_tilde_is / 3) * sinh( (1/3) * asinh( ((3*q_
```

t1_pre_split_p_larger_than_0 =

$$\frac{\sqrt{12}\,\sinh\left(\frac{\operatorname{asinh}\left(\frac{3\sqrt{3}\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)\left(4\sin(\phi)-2\widetilde{\Delta}_f\widetilde{\Delta}_\kappa\right)\sqrt{-\frac{1}{\cos(\phi)+\frac{\widetilde{\Delta}_f^{\,2}}{4}-\frac{\widetilde{\Delta}_\kappa^{\,2}}{4}-\frac{\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)^2}{4}}}}{8\left(2\cos(\phi)+\frac{\widetilde{\Delta}_f^{\,2}}{2}-\frac{\widetilde{\Delta}_\kappa^{\,2}}{2}-\frac{\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)^2}{2}\right)}\right)}{3}\right)\sqrt{\widetilde{\Delta}}}{6}$$

```
t1_pre_split_p_less_than_0 = simplify(-2 * (abs(q_tilde_is)/q_tilde_is) * sqrt(-p_tilde_is/3) *
```

t1_pre_split_p_less_than_0 =

$$-\frac{2\left|\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)\sigma_1\right|\cosh\left(\frac{\operatorname{acosh}\left(3\sqrt{3}\left|\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right|\left|\sigma_1\right|\left(\frac{1}{\widetilde{\Delta}_f^{\,2}-2\widetilde{\Delta}_\kappa^{\,2}+2\widetilde{\Delta}_\kappa\widetilde{\kappa}_c-\widetilde{\kappa}_c^{\,2}+4\cos(\phi)}\right)^{3/2}\right)}{3}\right)\sqrt{3}}{3\left(\widetilde{\Delta}_\kappa-\widetilde{\kappa}_c\right)\left(4\sin(\phi)-2\widetilde{\Delta}_f\widetilde{\Delta}_\kappa\right)}$$

where

$$\sigma_1 = 2\sin(\phi)-\widetilde{\Delta}_f\widetilde{\Delta}_\kappa$$

This is a complicated expression, since it's piecewise dependent on the sign of $\widetilde{p}$. We will tread carefully here.

Nuisance Splitting Strength - Direct Analysis Approach (Eqs. 25-26)

First, check $\phi = 0$

```
simplify(subs(p_tilde_is, [Delta_tilde_f, phi, Delta_tilde_kappa], [delta_tilde, 0, primary_tpd
```

ans =

$$-\frac{\widetilde{\delta}^2}{4}$$

We know that p will certainly be negative.

```
assumeAlso(delta_tilde ~= 0)
assumeAlso(delta_tilde, 'real')

% try to simplify this
assumeAlso(kappa_tilde_c < 2)
nuisance_tpd_phi_0 = simplify(subs(t1_pre_split_p_less_than_0, [Delta_tilde_f, phi, Delta_tilde
expand(simplify(series(nuisance_tpd_phi_0, delta_tilde, 0, 'Order', 1), 'Steps', 50))
```

ans =

$$\frac{|\tilde{\delta}|^{4/3}\left(4-\tilde{\kappa}_c^{\,2}\right)^{1/3}}{2\,\tilde{\delta}}+\frac{\tilde{\delta}\,|\tilde{\delta}|^{2/3}}{6\left(4-\tilde{\kappa}_c^{\,2}\right)^{1/3}}$$

```
assume(kappa_tilde_c, 'clear')
assume(delta_tilde, 'clear')
```

We can simplify $\frac{|\tilde{\delta}|^{\frac{4}{3}}}{\delta} \to \operatorname{sgn}\left(\tilde{\delta}\right)|\tilde{\delta}|^{\frac{1}{3}}$

Now, what happens for kappa_tilde_c between 2 and 2sqrt(2)?

```
assume(kappa_tilde_c, 'real')
assume(delta_tilde, 'real')
assumeAlso(delta_tilde ~= 0)
assumeAlso(kappa_tilde_c > 2 & kappa_tilde_c < sqrt(8))
expand(simplify(series(nuisance_tpd_phi_0, delta_tilde, 0, 'Order', 1), 'Steps', 50))
```

```
ans =
```

$$\frac{(-1)^{1/3}\,|\tilde{\delta}|^{4/3}\left(\tilde{\kappa}_c^{\,2}-4\right)^{1/3}}{2\,\tilde{\delta}}-\frac{(-1)^{2/3}\,\tilde{\delta}\,|\tilde{\delta}|^{2/3}}{6\left(\tilde{\kappa}_c^{\,2}-4\right)^{1/3}}$$

```
assume(kappa_tilde_c, 'clear')
```

This can be simplified by placing absolute value -> $|\tilde{\kappa}_c^2-4|^{\frac{1}{3}}$

However,

What happens exactly at $\tilde{\kappa}_c = 2$

```
nuisance_tpd_phi_0_kc_2 = subs(nuisance_tpd_phi_0, kappa_tilde_c, 2)
```

```
nuisance_tpd_phi_0_kc_2 =
```

$$\frac{\tilde{\delta}}{2}$$

We see that the cube root scaling is completely removed, and instead replaced with $\frac{\tilde{\delta}}{2}$

We discuss the implications of this more in the Robust TPD section.

Now, check $\phi = \pi$

```
simplify(subs(p_tilde_is, [Delta_tilde_f, phi, Delta_tilde_kappa], [primary_tpd_phi_pi(2), pi,
```

ans =

$$\frac{\widetilde{\delta}\left(\widetilde{\delta}-\widetilde{\kappa}_c\right)}{2}$$

Now that we know the sign of p, we know which hyperbolic equation to use

This expression is negative, as long as $0 < \widetilde{\delta} < \widetilde{\kappa_c}$

First, handle this case

```
assume(kappa_tilde_c, 'real')
assumeAlso(kappa_tilde_c > 0)
assumeAlso(delta_tilde ~= 0)
assumeAlso(delta_tilde, 'real')
%assumeAlso(delta_tilde > 0 & delta_tilde < kappa_tilde_c)
assumeAlso(delta_tilde < kappa_tilde_c)

% use the p > 0 equation
nuisance_tpd_phi_pi = simplify(subs(t1_pre_split_p_less_than_0, [Delta_tilde_f, phi, Delta_tilde
expand(simplify(series(nuisance_tpd_phi_pi, delta_tilde, 0, 'Order', 1), 'Steps', 50))
```

ans =

$$\begin{cases} -\sigma_2-\sigma_1 & \text{if } 0<\widetilde{\delta} \\ \sigma_2+\sigma_1 & \text{if } \widetilde{\delta}<0 \end{cases}$$

where

$$\sigma_1=\frac{2^{2/3}\,\widetilde{\delta}^{2/3}\,\widetilde{\kappa}_c^{\,2/3}}{6\left(\widetilde{\kappa}_c^{\,2}+4\right)^{1/6}}$$

$$\sigma_2=\frac{2^{1/3}\,\widetilde{\delta}^{1/3}\,\widetilde{\kappa}_c^{\,1/3}\left(\widetilde{\kappa}_c^{\,2}+4\right)^{1/6}}{2}$$

```
assumptions(delta_tilde)
```

ans =

$$\left(\widetilde{\delta}<\widetilde{\kappa}_c \quad \widetilde{\delta}\in\mathbb{R} \quad \widetilde{\kappa}_c\in\mathbb{R} \quad \widetilde{\delta}\neq 0 \quad 0<\widetilde{\kappa}_c\right)$$

```
% Is there a chance for a robust phi = pi TPD at kappa_c = 0?
nuisance_tpd_phi_pi_kc_0 = simplify(subs(nuisance_tpd_phi_pi, kappa_tilde_c, 0), 'Steps', 50)
```

nuisance_tpd_phi_pi_kc_0 =

$$-\frac{\sqrt{6}\ \sqrt{-\widetilde{\delta}}\ \cosh\left(\frac{\operatorname{acosh}\left(\frac{3\ \sqrt{6}\ |\widetilde{\delta}|\ \left(\frac{1}{\widetilde{\delta}}\right)^{3/2}\ \sqrt{-\frac{1}{\widetilde{\delta}}}}{2}\right)}{3}\right)|\widetilde{\delta}|}{3\ \sqrt{\widetilde{\delta}}}$$

```
expand(simplify(series(nuisance_tpd_phi_pi_kc_0, delta_tilde, 0, 'Order', 1, 'Direction', 'left
```

ans =

$$\frac{(-1)^{1/6}\,4^{2/3}\left(-\widetilde{\delta}\right)^{4/3}\,\mathrm{i}}{12}-\frac{(-1)^{5/6}\,4^{1/3}\left(-\widetilde{\delta}\right)^{2/3}\,\mathrm{i}}{2}$$

```
% No, it's still ^2/3

assume(delta_tilde, 'clear')
```

However, for $\widetilde{\delta}<0$, this expression is POSITIVE

```
assumeAlso(delta_tilde ~= 0)
assumeAlso(delta_tilde, 'real')
%assumeAlso(delta_tilde < 0)

assumptions(delta_tilde)
```

ans =

$$\left(\widetilde{\delta}\in\mathbb{R}\quad\widetilde{\delta}\neq 0\right)$$

```
nuisance_tpd_phi_pi = simplify(subs(t1_pre_split_p_larger_than_0, [Delta_tilde_f, phi, Delta_ti
% NOTE: This has to be "direction left" because the equation is ONLY valid
% for delta_tilde -> 0^-
expand(simplify(series(nuisance_tpd_phi_pi, delta_tilde, 0, 'Order', 1, 'Direction', 'left'),
```

ans =

$$-\frac{2^{1/3}\ \sqrt{\widetilde{\delta}}\ \widetilde{\kappa}_c^{\,1/3}\left(\widetilde{\kappa}_c^{\,2}+4\right)^{1/6}\,\mathrm{i}}{2\left(-\widetilde{\delta}\right)^{1/6}}+\frac{2^{2/3}\left(-\widetilde{\delta}\right)^{1/6}\ \sqrt{\widetilde{\delta}}\ \widetilde{\kappa}_c^{\,2/3}\,\mathrm{i}}{6\left(\widetilde{\kappa}_c^{\,2}+4\right)^{1/6}}$$

```

```

This result is written weird, but, there are a few things to notice.

First, the i in the numerator is not a problem. Since $\widetilde{\delta}<0$, the $\sqrt{\widetilde{\delta}}$ in the numerator cancels out the additional i.

Furthermore, we see that $\frac{\sqrt{\tilde{\delta}}}{\left(-\tilde{\delta}\right)^{\frac{1}{6}}} = \left(-\tilde{\delta}\right)^{\frac{1}{3}}$. This is just accounting for $\tilde{\delta}$ being assumed negative here. The scaling rate is still the same.

Thus, we can replace the treatment of $\pm\tilde{\delta}$ with just $|\tilde{\delta}|$. The negative sign out front can be handled using sgn

```
% Is there a chance for a robust phi = pi TPD at kappa_c = 0?
nuisance_tpd_phi_pi_kc_0 = simplify(subs(nuisance_tpd_phi_pi, kappa_tilde_c, 0), 'Steps', 50)
```

```
nuisance_tpd_phi_pi_kc_0 =
```

$$\frac{\sqrt{6}\,\sinh\left(\frac{\operatorname{asinh}\left(\frac{3\sqrt{6}}{2\,|\tilde{\delta}|}\right)}{3}\right)|\tilde{\delta}|}{3}$$

```
expand(simplify(series(nuisance_tpd_phi_pi_kc_0, delta_tilde, 0, 'Order', 1, 'Direction', 'left
```

```
ans =
```

$$\frac{2^{2/3}\left(-\tilde{\delta}\right)^{2/3}}{2} - \frac{2^{2/3}\,4^{1/3}\left(-\tilde{\delta}\right)^{4/3}}{12}$$

```
% No, it's still ^2/3
assume(delta_tilde, 'clear')
```

Robust Transmission Peak Degeneracies

Critically, we notice that for $\phi = 0, \tilde{\kappa_c} = 2$, the nuisance scaling is $\frac{\tilde{\delta}}{2}$, instead of cube root scaling, while the peak splitting still retains $\tilde{\Delta_\nu} \propto \sqrt{\epsilon}$!

This can be understood through the `disc=0` contour. Normally, this contour is described by a 6th-degree polynomial, which forms "cusps" at TPDs. These cusps contribute to the nuisance parameter hypersensitivity. However, at the Robust TPD, we have:

```
robust_disc = simplify(subs(disc == 0, [kappa_tilde_c, phi], [2 0]), 'Steps', 50)
```

```
robust_disc =
```

$$\tilde{\Delta}_f^{\,2} + 16\,\tilde{\Delta}_\kappa = 8\,\tilde{\Delta}_\kappa^{\,2} \vee \tilde{\Delta}_f^{\,2} + \tilde{\Delta}_\kappa^{\,2} = 2\,\tilde{\Delta}_\kappa$$

The disc contour factorizes! The 6th degree polynomial becomes the union of a circle and a hyperbola.

In all other cases, the disc == 0 contour forms sharp "cusps" at TPDs.

```
disc_0 = expand(disc == 0)
```

```
disc_0 =
```

$$\frac{\widetilde{\Delta}_f^{\,6}}{16} - \frac{3\,\widetilde{\Delta}_f^{\,4}\,\widetilde{\Delta}_\kappa^{\,2}}{8} + \frac{3\,\widetilde{\Delta}_f^{\,4}\,\widetilde{\Delta}_\kappa\,\widetilde{\kappa}_c}{8} - \frac{3\,\widetilde{\Delta}_f^{\,4}\,\widetilde{\kappa}_c^{\,2}}{16} + \frac{3\,\widetilde{\Delta}_f^{\,4}\cos(\phi)}{4} - \frac{15\,\widetilde{\Delta}_f^{\,2}\,\widetilde{\Delta}_\kappa^{\,4}}{16} + \frac{15\,\widetilde{\Delta}_f^{\,2}\,\widetilde{\Delta}_\kappa^{\,3}\,\widetilde{\kappa}_c}{8} - \frac{3\,\widetilde{\Delta}_f^{\,2}\,\widetilde{\Delta}_\kappa^{\,2}\,\widetilde{\kappa}_c^{\,2}}{16}$$

Appendix

EP Hypersensitivity (Appendix C.A) (Eq. 39)

```
iss = imag(spec_split)
```

```
iss =
```

$$\operatorname{imag}\left(\sqrt{-\widetilde{\Delta}_f^{\,2} + 2\,\widetilde{\Delta}_f\,\widetilde{\Delta}_\kappa\,\mathrm{i} + \widetilde{\Delta}_\kappa^{\,2} - 4\,\mathrm{e}^{\phi\,\mathrm{i}}}\right)$$

First, we consider $\phi = 0$, with the EP at $\widetilde{\Delta}_\kappa = +2$, and $\widetilde{\Delta}_f = 0$

In this case, the optimal sensing path through the EP substitues $\widetilde{\Delta}_\kappa = 2 - \widetilde{\epsilon}$, and nuisance perturbations manifest as $\widetilde{\Delta}_f = \widetilde{\delta}$

The other EP in this case is at $\widetilde{\Delta}_\kappa = -2$, with $+\widetilde{\epsilon}$

```
right_ep = subs(iss, [Delta_tilde_kappa, Delta_tilde_f, phi], [2 - epsilon_tilde, delta_tilde,
```

```
right_ep =
```

$$\operatorname{imag}\left(\sqrt{\left(\widetilde{\varepsilon} - 2\right)^2 - \widetilde{\delta}^2 - 4 - 2\,\widetilde{\delta}\,\left(\widetilde{\varepsilon} - 2\right)\mathrm{i}}\right)$$

```
left_ep = subs(iss, [Delta_tilde_kappa, Delta_tilde_f, phi], [-2 + epsilon_tilde, delta_tilde,
```

```
left_ep =
```

$$\operatorname{imag}\left(\sqrt{\left(\widetilde{\varepsilon} - 2\right)^2 - \widetilde{\delta}^2 - 4 + 2\,\widetilde{\delta}\,\left(\widetilde{\varepsilon} - 2\right)\mathrm{i}}\right)$$

First, both EPs with no nuisance

```
perfect_right = subs(right_ep, delta_tilde, 0)
```

```
perfect_right =
```

$$\operatorname{imag}\left(\sqrt{\left(\widetilde{\varepsilon} - 2\right)^2 - 4}\right)$$

Check the susceptibility, the derivative as $\widetilde{\epsilon} \to 0^+$

```
limit(diff(perfect_right, epsilon_tilde), epsilon_tilde, 0, "right")
```

```
ans = ∞
```

For the left EP:

```
perfect_left = subs(left_ep, delta_tilde, 0)
```

```
perfect_left =
```

$$\mathrm{imag}\left(\sqrt{\left(\tilde{\varepsilon}-2\right)^2-4}\right)$$

```
limit(diff(perfect_left, epsilon_tilde), epsilon_tilde, 0, "right")
```

```
ans = ∞
```

Next, we look at the same result for nuisance-only perturbations

```
right_ep_nuisance = subs(right_ep, epsilon_tilde, 0)
```

```
right_ep_nuisance =
```

$$\mathrm{imag}\left(\sqrt{-\tilde{\delta}^2+4\,\tilde{\delta}\,\mathrm{i}}\right)$$

```
left_ep_nuisance = subs(left_ep, epsilon_tilde, 0)
```

```
left_ep_nuisance =
```

$$\mathrm{imag}\left(\sqrt{-\tilde{\delta}^2-4\,\tilde{\delta}\,\mathrm{i}}\right)$$

```
limit(diff(right_ep_nuisance, delta_tilde), delta_tilde, 0)
```

```
ans = ∞
```

```
limit(diff(left_ep_nuisance, delta_tilde), delta_tilde, 0)
```

```
ans = −∞
```

The susceptibility is ∞ to both sensing and nuisance perturbations at EPs.

Finally, we explore what happens to the susceptibility with respect to the sensing parameter, in the presence of $\tilde{\delta} \neq 0$

```
limit(diff(right_ep, epsilon_tilde), epsilon_tilde, 0, "right")
```

```
ans =
```

$$-\frac{\operatorname{imag}\left(\frac{4+2\,\widetilde{\delta}\,\mathrm{i}}{\sqrt{-\widetilde{\delta}^2+4\,\widetilde{\delta}\,\mathrm{i}}}\right)}{2}$$

```
limit(diff(left_ep, epsilon_tilde), epsilon_tilde, 0, "right")
```

ans =

$$\frac{\operatorname{imag}\left(\frac{-4+2\,\widetilde{\delta}\,\mathrm{i}}{\sqrt{-\widetilde{\delta}^2-4\,\widetilde{\delta}\,\mathrm{i}}}\right)}{2}$$

Which is not infinity. Thus, the "diverging signal response" characteristic of EPs vanishes in the presence of any nuisance parameter.

These results can be applied to EPs at any ϕ

Residual Splitting at a TED (Appendix D.A) (Eqs. 41-46)

First, we solve the Vieta equations that relate the cubic to the sum and product of its roots. At an imperfect TPD (that still crosses the discriminant), two of the roots are repeated r, and one is single s

```
syms r s p_tilde q_tilde
rel1 = 2 * r + s == 0;
rel2 = r^2 + r * s + r * s == p_tilde;
rel3 = r * r * s == -q_tilde;
whatis_s = solve(rel1, s);
% Now that we have s, sub into the other relation
rel2 = subs(rel2, s, whatis_s);
rel3 = simplify(subs(rel3, s, whatis_s));
q_is = solve(rel3, r)
```

q_is =

$$\begin{pmatrix} \frac{2^{2/3}\,\widetilde{q}^{1/3}}{2} \\ \frac{2^{2/3}\,\widetilde{q}^{1/3}\left(-\frac{1}{2}+\frac{\sqrt{3}\,\mathrm{i}}{2}\right)}{2} \\ -\frac{2^{2/3}\,\widetilde{q}^{1/3}\left(\frac{1}{2}+\frac{\sqrt{3}\,\mathrm{i}}{2}\right)}{2} \end{pmatrix}$$

```
p_is = solve(rel2, r)
```

p_is =

$$\begin{pmatrix} -\frac{\sqrt{3}\,\sqrt{-\tilde{p}}}{3} \\ \frac{\sqrt{3}\,\sqrt{-\tilde{p}}}{3} \end{pmatrix}$$

```
% ignore the complex solutions - q is real here
q_is = q_is(1);
% distance between the roots
dist = abs(s - r);
dist = subs(dist, s, whatis_s);
optimal_path_err = simplify(subs(dist, r, q_is))
```

optimal_path_err =

$$\frac{3\,2^{2/3}\,|\tilde{q}|^{1/3}}{2}$$

```
optimal_path_err_full = subs(optimal_path_err, q_tilde, q_tilde_is);
```

We see that min splitting is exactly based on $|\tilde{q}|^{1/3}$

```
assume(kappa_tilde_c, 'clear');
assume(kappa_tilde_c, 'real'); assumeAlso(kappa_tilde_c > 0);
assumptions(kappa_tilde_c)
```

ans =

$$\left(\tilde{\kappa}_c \in \mathbb{R} \quad 0 < \tilde{\kappa}_c\right)$$

```
min_split_phi_0 = simplify(subs(optimal_path_err_full, [phi, Delta_tilde_kappa], [0, primary_tp
```

min_split_phi_0 =

$$\frac{3\,|\tilde{\Delta}_f|^{1/3}\left|\frac{\sqrt{8-\tilde{\kappa}_c^{\,2}}}{2}-\frac{\tilde{\kappa}_c}{2}\right|^{1/3}\left|\frac{\tilde{\kappa}_c}{2}+\frac{\sqrt{8-\tilde{\kappa}_c^{\,2}}}{2}\right|^{1/3}}{2}$$

```
assumeAlso(kappa_tilde_c < 2)
assumptions(kappa_tilde_c);
expand(simplify(min_split_phi_0, 'Steps', 500))
```

ans =

$$\frac{3\,2^{2/3}\left(4-\tilde{\kappa}_c^{\,2}\right)^{1/3}\left(\tilde{\Delta}_f^{\,2}\right)^{1/6}}{4}$$

```
assume(kappa_tilde_c, 'clear')
assume(kappa_tilde_c, 'real'); assumeAlso(kappa_tilde_c >= 2 & kappa_tilde_c < sqrt(8))
assumptions(kappa_tilde_c);
simplify(min_split_phi_0, 'Steps', 809)
```

```
ans =
```

$$\frac{3\,2^{2/3}\left(\tilde{\kappa}_c^{\,2}-4\right)^{1/3}\left(\tilde{\Delta}_f^{\,2}\right)^{1/6}}{4}$$

Squaring first, and then taking to the 1/6 is just a careful way of writing absolute value cube root. Also the flipping of the sign just means that we need absolute value in the kc^2 - 4

```
assume(kappa_tilde_c, 'clear'); assume(kappa_tilde_c, 'real'); assumeAlso(kappa_tilde_c > 0);
min_split_phi_pi = simplify(subs(optimal_path_err_full, [phi, Delta_tilde_f], [pi, primary_tpd_
```

```
min_split_phi_pi =
```

$$\frac{3\left|\tilde{\Delta}_\kappa\left(\tilde{\Delta}_\kappa-\tilde{\kappa}_c\right)\right|^{1/3}\left(\tilde{\kappa}_c^{\,2}+4\right)^{1/6}}{2}$$

We need to Puiseux expand this

```
% For a small perturbation with delta < kappa_tilde_c
assumeAlso(Delta_tilde_kappa < kappa_tilde_c)
min_split_phi_pi_expand = (expand(series(min_split_phi_pi, Delta_tilde_kappa, 0, 'Order', 1)))
```

```
min_split_phi_pi_expand =
```

$$\frac{3\,\tilde{\kappa}_c^{\,1/3}\left(\tilde{\kappa}_c^{\,2}+4\right)^{1/6}\left(\frac{\tilde{\Delta}_\kappa}{\operatorname{sign}\left(\tilde{\Delta}_\kappa\right)}\right)^{1/3}}{2}$$

Matlab prefers to divide by sign, but that's just absolute value

Cusp Geometry and TPD vs TED Location (Appendix D.C) (Eqs. 57-58)

Here, we seek to bound the distance between the TED location in detuning space versus the TPD location. Here are the variables used:

$\tilde{\delta}$ - Differential nuisance perturbation (same as above), where $\tilde{\delta}\cdot\tilde{\epsilon}=0$

$\tilde{\mu}$ - Corresponding differential in the "sensing" coordinate location of the TED as it moves

$\tilde{\epsilon}$ - Sensing perturbation that crosses the TED (used later to show that TEDs retain square root splitting)

```
assume(delta_tilde, 'real')
assumeAlso(delta_tilde ~= 0)
syms mu_tilde real
assumeAlso(mu_tilde ~= 0)
```

```
assumeAlso(kappa_tilde_c < sqrt(8))

disc_0_phi_0 = simplify(subs(disc_0, phi, 0));

%simplify(subs(disc_0_phi_0, [Delta_tilde_kappa, Delta_tilde_f], [primary_tpd_phi_0(1), 0]), 'S
%ftd = simplify(subs(disc_0_phi_0, [Delta_tilde_kappa, Delta_tilde_f], [primary_tpd_phi_0(1) +
%subs(ftd, kappa_tilde_c, 2)

p_tilde_TED_phi_0 = simplify(subs(p_tilde_is, [Delta_tilde_f, Delta_tilde_kappa, phi], [delta_t
```

p_tilde_TED_phi_0 =

$$\frac{\tilde{\mu}^2}{2} - \frac{\tilde{\delta}^2}{4} - \frac{\tilde{\mu}\sqrt{8-\tilde{\kappa}_c^2}}{2}$$

```
q_tilde_TED_phi_0 = simplify(subs(q_tilde_is, [Delta_tilde_f, Delta_tilde_kappa, phi], [delta_t
```

q_tilde_TED_phi_0 =

$$\frac{\tilde{\delta}\left(2\tilde{\mu}\sqrt{8-\tilde{\kappa}_c^2} + \tilde{\kappa}_c^2 - 2\tilde{\mu}^2 - 4\right)}{8}$$

```
ted_condition_phi_0 = -4 * p_tilde_TED_phi_0^3 - 27 * q_tilde_TED_phi_0^2 == 0
```

ted_condition_phi_0 =

$$4\left(\frac{\tilde{\mu}\sqrt{8-\tilde{\kappa}_c^2}}{2} + \frac{\tilde{\delta}^2}{4} - \frac{\tilde{\mu}^2}{2}\right)^3 - \frac{27\tilde{\delta}^2\left(2\tilde{\mu}\sqrt{8-\tilde{\kappa}_c^2} + \tilde{\kappa}_c^2 - 2\tilde{\mu}^2 - 4\right)^2}{64} = 0$$

```
ted_approx = taylor(lhs(ted_condition_phi_0), [mu_tilde, delta_tilde], 'Order', 4)
```

ted_approx =

$$\frac{\tilde{\mu}^3\left(8-\tilde{\kappa}_c^2\right)^{3/2}}{2} - \frac{27\tilde{\delta}^2\left(\tilde{\kappa}_c^2-4\right)^2}{64} - \frac{27\tilde{\delta}^2\tilde{\mu}\left(\tilde{\kappa}_c^2-4\right)\sqrt{8-\tilde{\kappa}_c^2}}{16}$$

This is a classic "cusp" scaling that we see, where $\tilde{\mu} \sim \tilde{\delta}^{\frac{2}{3}}$

```
ted_approx_implies_0 = simplify((1/2) * mu_tilde^3 * (8 - kappa_tilde_c^2)^(3/2) == (27/64) * d
```

ted_approx_implies_0 =

$$32\tilde{\mu}^3\left(8-\tilde{\kappa}_c^2\right)^{3/2} = 27\tilde{\delta}^2\left(\tilde{\kappa}_c^2-4\right)^2$$

```
mu_tilde_is = solve(ted_approx_implies_0, mu_tilde, 'ReturnConditions', true)
```

mu_tilde_is = *struct with fields:*

```
      mu_tilde: [3×1 sym]
    parameters: [1×0 sym]
    conditions: [3×1 sym]
```

```
assumeAlso(kappa_tilde_c < sqrt(8))

simplify(mu_tilde_is.mu_tilde, 'Steps', 100)
```

ans =

$$\begin{pmatrix} \frac{3\,2^{1/3}\,\widetilde{\delta}^{2/3}\,\sigma_1\,\sigma_2}{4\,\sqrt{8-\widetilde{\kappa}_c^{\,2}}} \\ \frac{3\,2^{1/3}\,\widetilde{\delta}^{2/3}\,\left(-1+\sqrt{3}\,\mathrm{i}\right)\sigma_1\,\sigma_2}{\sigma_3} \\ -\frac{3\,2^{1/3}\,\widetilde{\delta}^{2/3}\,\left(1+\sqrt{3}\,\mathrm{i}\right)\sigma_1\,\sigma_2}{\sigma_3} \end{pmatrix}$$

where

$$\sigma_1 = \left(\widetilde{\kappa}_c - 2\right)^{2/3}$$

$$\sigma_2 = \left(\widetilde{\kappa}_c + 2\right)^{2/3}$$

$$\sigma_3 = 8\,\sqrt{8-\widetilde{\kappa}_c^{\,2}}$$

```
simplify(mu_tilde_is.conditions, 'Steps', 100)
```

ans =

$$\begin{pmatrix} \widetilde{\delta}^{2/3}\,\sigma_1 \in \mathbb{R} \wedge \widetilde{\kappa}_c \neq 2 \\ \widetilde{\delta}^{2/3}\left(-\frac{1}{2}+\frac{\sqrt{3}\,\mathrm{i}}{2}\right)\sigma_1 \in \mathbb{R} \wedge \widetilde{\kappa}_c \neq 2 \\ \widetilde{\delta}^{2/3}\left(\frac{1}{2}+\frac{\sqrt{3}\,\mathrm{i}}{2}\right)\sigma_1 \in \mathbb{R} \wedge \widetilde{\kappa}_c \neq 2 \end{pmatrix}$$

where

$$\sigma_1 = \left(\widetilde{\kappa}_c - 2\right)^{2/3}$$

```
mu_scaling_2_3 = simplify(abs(mu_tilde_is.mu_tilde(1)), 'Steps', 100)
```

mu_scaling_2_3 =

$$\frac{3\,2^{1/3}\,|\tilde{\kappa}_c-2|^{2/3}\,|\tilde{\delta}|^{2/3}\,(\tilde{\kappa}_c+2)^{2/3}}{4\,\sqrt{8-\tilde{\kappa}_c^{\,2}}}$$

Now, do $\phi = \pi$

```
assume(kappa_tilde_c, 'clear'); assume(kappa_tilde_c, 'real'); assumeAlso(kappa_tilde_c > 0);
disc_0_phi_pi = simplify(subs(disc_0, phi, pi))
```

disc_0_phi_pi =

$$\tilde{\Delta}_f^{\,6}+6\,\tilde{\Delta}_f^{\,4}\,\tilde{\Delta}_\kappa\,\tilde{\kappa}_c+30\,\tilde{\Delta}_f^{\,2}\,\tilde{\Delta}_\kappa^{\,3}\,\tilde{\kappa}_c+48\,\tilde{\Delta}_f^{\,2}\,\tilde{\Delta}_\kappa^{\,2}+3\,\tilde{\Delta}_f^{\,2}\,\tilde{\kappa}_c^{\,4}+24\,\tilde{\Delta}_f^{\,2}\,\tilde{\kappa}_c^{\,2}+48\,\tilde{\Delta}_f^{\,2}+24\,\tilde{\Delta}_\kappa^{\,5}\,\tilde{\kappa}_c+32\,\tilde{\Delta}_\kappa^{\,3}$$

```
p_tilde_TED_phi_pi = simplify(subs(p_tilde_is, [Delta_tilde_kappa, Delta_tilde_f, phi], [delta_
```

p_tilde_TED_phi_pi =

$$\frac{\tilde{\delta}^2}{4}-\frac{\left(\tilde{\mu}+\sqrt{\tilde{\kappa}_c^{\,2}+4}\right)^2}{4}+\frac{\left(\tilde{\delta}-\tilde{\kappa}_c\right)^2}{4}+1$$

```
q_tilde_TED_phi_pi = simplify(subs(q_tilde_is, [Delta_tilde_kappa, Delta_tilde_f, phi], [delta_
```

q_tilde_TED_phi_pi =

$$-\frac{\tilde{\delta}\,\left(\tilde{\mu}+\sqrt{\tilde{\kappa}_c^{\,2}+4}\right)\left(\tilde{\delta}-\tilde{\kappa}_c\right)}{4}$$

```
ted_condition_phi_pi = -4 * p_tilde_TED_phi_pi^3 - 27 * q_tilde_TED_phi_pi^2 == 0
```

ted_condition_phi_pi =

$$-4\left(\frac{\tilde{\delta}^2}{4}-\frac{\left(\tilde{\mu}+\sqrt{\tilde{\kappa}_c^{\,2}+4}\right)^2}{4}+\frac{\left(\tilde{\delta}-\tilde{\kappa}_c\right)^2}{4}+1\right)^3-\frac{27\,\tilde{\delta}^2\,\left(\tilde{\mu}+\sqrt{\tilde{\kappa}_c^{\,2}+4}\right)^2\left(\tilde{\delta}-\tilde{\kappa}_c\right)^2}{16}=0$$

```
ted_approx_phi_pi = taylor(lhs(ted_condition_phi_pi), [mu_tilde, delta_tilde], 'Order', 4)
```

ted_approx_phi_pi =

$$\tilde{\delta}^3\left(\frac{\tilde{\kappa}_c^{\,3}}{2}+\frac{27\,\tilde{\kappa}_c\left(\tilde{\kappa}_c^{\,2}+4\right)}{8}\right)-\frac{27\,\tilde{\delta}^2\,\tilde{\kappa}_c^{\,2}\left(\tilde{\kappa}_c^{\,2}+4\right)}{16}+\tilde{\delta}\,\tilde{\mu}^2\left(2\,\tilde{\kappa}_c\,\sigma_1+\tilde{\kappa}_c\left(\tilde{\kappa}_c^{\,2}+4\right)\right)+2\,\tilde{\mu}^3\,\sigma_2\,\sigma_1-\frac{1}{}$$

where

$$\sigma_1=\frac{\tilde{\kappa}_c^{\,2}}{4}+1$$

$$\sigma_2=\sqrt{\tilde{\kappa}_c^{\,2}+4}$$

The only terms that we are going to take from this are the $\widetilde{\delta^2}$ and $\widetilde{\mu}^3$ terms. Although, it is interesting that this expansion contains a $\widetilde{\delta^3}$ term, while $\phi = 0$ did not. Thus, the corrections for $\phi = \pi$ come into play "sooner" than for $\phi = 0$. This makes sense looking at the Fig 2 plots.

```
sigma1 = sqrt(kappa_tilde_c^2 + 4);
sigma2 = (kappa_tilde_c^2)/4 + 1;
ted_approx_implies_pi = delta_tilde^2 * (kappa_tilde_c^2 * 27/16) * (kappa_tilde_c^2 + 4) == 2
```

```
ted_approx_implies_pi =
```

$$\frac{27\,\widetilde{\delta}^2\,\widetilde{\kappa}_c^{\,2}\left(\widetilde{\kappa}_c^{\,2}+4\right)}{16} = 2\,\widetilde{\mu}^3\,\sqrt{\widetilde{\kappa}_c^{\,2}+4}\left(\frac{\widetilde{\kappa}_c^{\,2}}{4}+1\right)$$

```
mu_tilde_is_pi = solve(ted_approx_implies_pi, mu_tilde, 'ReturnConditions', true)
```

```
mu_tilde_is_pi = struct with fields:
      mu_tilde: [3×1 sym]
    parameters: [1×0 sym]
    conditions: [3×1 sym]
```

```
simplify(abs(mu_tilde_is_pi.mu_tilde), 'Steps', 100)
```

```
ans =
```

$$\begin{pmatrix}\sigma_1\\ \sigma_1\\ \sigma_1\end{pmatrix}$$

where

$$\sigma_1 = \frac{3\,\widetilde{\kappa}_c^{\,2/3}\left(\widetilde{\delta}^2\right)^{1/3}}{2\left(\widetilde{\kappa}_c^{\,2}+4\right)^{1/6}}$$

```
simplify(mu_tilde_is_pi.conditions, 'Steps', 100)
```

```
ans =
```

$$\begin{pmatrix} 0 < \widetilde{\delta} \\ \widetilde{\delta}^{2/3}\left(-\frac{1}{2}+\frac{\sqrt{3}\,\mathrm{i}}{2}\right) \in \mathbb{R} \wedge \widetilde{\delta} < 0 \\ \widetilde{\delta}^{2/3}\left(\frac{1}{2}+\frac{\sqrt{3}\,\mathrm{i}}{2}\right) \in \mathbb{R} \wedge \widetilde{\delta} < 0 \end{pmatrix}$$

TEDs retain square root splitting (Appendix D.B) (Eqs. 47-56)

```
%  Goal: Show directly from the closed-form root expressions
%
```

```
%      theta      = (1/3) * arccos( (3 q)/(2 p) * sqrt(-3/p) )
%      nu_+       = 2 sqrt(-p/3) cos(theta)
%      eta        = 2 sqrt(-p/3) cos(theta - 2 pi/3)
%      nu_-       = 2 sqrt(-p/3) cos(theta - 4 pi/3)
%
%  that crossing a TED yields square-root peak splitting
```

First, we note that at any TED, the 3 roots can be parameterized using a single number, $r > 0$. At the TED, two roots are degenerate, located at $-r$, with a spectator root displaced at $2r$. (these are the Vieta relations)

This also means that $p^{\text{TED}}, q^{\text{TED}}$ have simple expressions.

Furthermore, the arccos argument in Eqn 13 is $Z = \frac{3q}{2p}\sqrt{\frac{-3}{p}}$, which equals 1 at a TED.

Given a sensing perturbation eps > 0, $p \approx p^{\text{TED}} + ||\epsilon|| p_1$, and similar for q, where p_1, q_1 are directional gradients along $\hat{\epsilon}$

How does this expansion of p, q propagate into the arccos argument (and thus into the location of the roots?)

```
%% ------------------------------------------------------------------
%  Step 1: Symbolic setup with assumptions
%  ------------------------------------------------------------------
%  r > 0 parameterizes the TED via Vieta. eps > 0 is the perturbation
%  magnitude. p_1, q_1 are directional gradients along eps_hat.
%  Z_1 < 0 is defined to select the physical branch (Disc > 0 side).

syms r positive
syms epsilon positive
syms Z_1 negative real
syms p_1 q_1 real

% Vieta at the TED: repeated root -r, spectator 2r (sum = 0).
%   p^TED = -3r^2, q^TED = -2r^3 (with our sign convention).
p_TED = -3*r^2
```

p_TED = $-3\,r^2$

```
q_TED = -2*r^3
```

q_TED = $-2\,r^3$

```

% Perturbed cubic coefficients
p_pert = p_TED + p_1*epsilon
```

p_pert = $\varepsilon\,p_1 - 3\,r^2$

```
q_pert = q_TED + q_1*epsilon
```

q_pert = $\varepsilon\, q_1 - 2\, r^3$

```
%% ------------------------------------------------------------------------
%  Step 2: Verify Z at the TED
%  ------------------------------------------------------------------------
Z_pert = (3*q_pert)/(2*p_pert) * sqrt(-3/p_pert)
```

Z_pert =

$$\frac{\sqrt{3}\ \sqrt{-\frac{1}{\varepsilon\, p_1 - 3\, r^2}}\ \left(3\,\varepsilon\, q_1 - 6\, r^3\right)}{2\,\varepsilon\, p_1 - 6\, r^2}$$

```
Z_at_TED = simplify(subs(Z_pert, epsilon, 0))
```

Z_at_TED = 1

```
% Expected: 1
```

Next, compute dZ/d(eps) at eps = 0. On the physical branch, we require this to be negative (Z decreases below 1 as we leave the TED).

```
dZ_deps_at_TED = simplify(subs(diff(Z_pert, epsilon), epsilon, 0))
```

dZ_deps_at_TED =

$$-\frac{q_1 - p_1\, r}{2\, r^3}$$

Write without r for the appdx

```
r_is_p = solve(p_TED == p_tilde, r)
```

Warning: Solutions are only valid under certain conditions. To include parameters and conditions in the solution, specify the 'ReturnConditions' value as 'true'.

r_is_p =

$$\frac{\sqrt{3}\ \sqrt{-\widetilde{p}}}{3}$$

```
simplify(subs(dZ_deps_at_TED, r, r_is_p), 'All', true)
```

ans =

$$\begin{pmatrix} -\frac{3\ \sqrt{3}\ q_1 - 3\, p_1\ \sqrt{-\widetilde{p}}}{2\ \left(-\widetilde{p}\right)^{3/2}} \\ -\frac{3\ \sqrt{3}\ \left(q_1 - \frac{\sqrt{3}\ p_1\ \sqrt{-\widetilde{p}}}{3}\right)}{2\ \left(-\widetilde{p}\right)^{3/2}} \end{pmatrix}$$

The "return condition" here is that $\widetilde{p} < 0$, which was already necessary.

Next:

```
% Physical branch condition: Z_1 = (q_1 - r*p_1)/(2*r^3) < 0.
```

The key here is that arccos(Z) has a branch point at Z=1. What happens with 1 + Z_1*eps to the arccos?

```
%  arccos(Z) has a branch point at Z=1. We DO NOT assert the expansion.
%  We substitute Z_physical = 1 + Z_1*eps and ask the CAS for a series
%  expansion in eps. The CAS correctly produces a Puiseux series
%  containing sqrt(eps), confirming the branch-point structure.
theta_arg = 1 + Z_1*epsilon;  % physical branch
```

Now, we see how this branch cut propagates into the root locations themselves

```
%  We build nu_plus, eta, nu_minus using the FULL acos argument from
%  Step 4, not an asserted leading-order substitution. Then the CAS
%  taylor-expands each root.

theta_full = acos(theta_arg) / 3;
prefactor = 2 * sqrt(-p_pert/3);
nu_plus_raw  = prefactor * cos(theta_full)
```

```
nu_plus_raw =
```

$$2\cos\left(\frac{\operatorname{acos}(Z_1\,\varepsilon+1)}{3}\right)\sqrt{r^2-\frac{\varepsilon\,p_1}{3}}$$

```
eta_raw      = prefactor * cos(theta_full - 2*sym(pi)/3)
```

```
eta_raw =
```

$$2\cos\left(\frac{2\,\pi}{3}-\frac{\operatorname{acos}(Z_1\,\varepsilon+1)}{3}\right)\sqrt{r^2-\frac{\varepsilon\,p_1}{3}}$$

```
nu_minus_raw = prefactor * cos(theta_full - 4*sym(pi)/3)
```

```
nu_minus_raw =
```

$$2\cos\left(\frac{4\,\pi}{3}-\frac{\operatorname{acos}(Z_1\,\varepsilon+1)}{3}\right)\sqrt{r^2-\frac{\varepsilon\,p_1}{3}}$$

```

%% ----------------------------------------------------------------------
%  Step 6: CAS-computed series expansions
%  ----------------------------------------------------------------------
%  Since each expression contains sqrt(eps) from the acos branch point,
%  we ask the CAS to expand to Order 2 in eps, which captures both
%  sqrt(eps) terms (leading) and eps^1 terms (subleading).

nu_plus_series  = series(nu_plus_raw,  epsilon, 'Order', 3);
```

```
eta_series      = series(eta_raw,      epsilon, 'Order', 3);
nu_minus_series = series(nu_minus_raw, epsilon, 'Order', 3);

%% ------------------------------------------------------------------------
%  Step 7: Root shifts from their TED values
%  ------------------------------------------------------------------------
Delta_nu_plus  = simplify(nu_plus_series  - 2*r)
```

Delta_nu_plus =

$$-\frac{\varepsilon\,(3\,p_1 - 2\,Z_1\,r^2)}{9\,r}$$

```
Delta_eta      = collect((eta_series      - (-r)), epsilon)
```

Delta_eta =

$$\left(\frac{p_1}{6\,r} - \frac{Z_1\,r}{9}\right)\varepsilon + \varepsilon^{3/2}\left(\frac{5\,\sqrt{2}\,\sqrt{3}\,(-Z_1)^{3/2}\,r}{324} - \frac{\sqrt{2}\,\sqrt{3}\,\sqrt{-Z_1}\,p_1}{18\,r}\right) + \frac{\sqrt{2}\,\sqrt{3}\,\sqrt{-Z_1}\,\sqrt{\varepsilon}\,r}{3}$$

```
Delta_nu_minus = collect((nu_minus_series - (-r)), epsilon)
```

Delta_nu_minus =

$$\left(\frac{p_1}{6\,r} - \frac{Z_1\,r}{9}\right)\varepsilon - \varepsilon^{3/2}\left(\frac{5\,\sqrt{2}\,\sqrt{3}\,(-Z_1)^{3/2}\,r}{324} - \frac{\sqrt{2}\,\sqrt{3}\,\sqrt{-Z_1}\,p_1}{18\,r}\right) - \frac{\sqrt{2}\,\sqrt{3}\,\sqrt{-Z_1}\,\sqrt{\varepsilon}\,r}{3}$$

```
% requires p < 0

% These square root coefficients are reported in the paper
subs(Delta_eta, r, r_is_p)
```

ans =

$$\varepsilon^{3/2}\left(\frac{5\,\sqrt{2}\,(-Z_1)^{3/2}\,\sqrt{-\tilde{p}}}{324} - \frac{\sqrt{2}\,\sqrt{-Z_1}\,p_1}{6\,\sqrt{-\tilde{p}}}\right) - \varepsilon\left(\frac{\sqrt{3}\,Z_1\,\sqrt{-\tilde{p}}}{27} - \frac{\sqrt{3}\,p_1}{6\,\sqrt{-\tilde{p}}}\right) + \frac{\sqrt{2}\,\sqrt{-Z_1}\,\sqrt{\varepsilon}\,\sqrt{-\tilde{p}}}{3}$$

```
subs(Delta_nu_plus, r, r_is_p)
```

ans =

$$-\frac{\sqrt{3}\,\varepsilon\left(3\,p_1 + \frac{2\,Z_1\,\tilde{p}}{3}\right)}{9\,\sqrt{-\tilde{p}}}$$

```
subs(Delta_nu_minus, r, r_is_p)
```

ans =

$$-\varepsilon^{3/2}\left(\frac{5\,\sqrt{2}\,(-Z_1)^{3/2}\,\sqrt{-\tilde{p}}}{324} - \frac{\sqrt{2}\,\sqrt{-Z_1}\,p_1}{6\,\sqrt{-\tilde{p}}}\right) - \varepsilon\left(\frac{\sqrt{3}\,Z_1\,\sqrt{-\tilde{p}}}{27} - \frac{\sqrt{3}\,p_1}{6\,\sqrt{-\tilde{p}}}\right) - \frac{\sqrt{2}\,\sqrt{-Z_1}\,\sqrt{\varepsilon}\,\sqrt{-\tilde{p}}}{3}$$

```
% Expected:
%   Delta_nu_plus  = O(eps) only          – LINEAR
%   Delta_eta      = +sqrt(6)*sqrt(Z_1)*r/3 * sqrt(eps) + O(eps)
%   Delta_nu_minus = -sqrt(6)*sqrt(Z_1)*r/3 * sqrt(eps) + O(eps)

%% ------------------------------------------------------------------
%  Step 8: Peak splitting and a_sqrt^TED
%  ------------------------------------------------------------------
%  Delta_nu^TED = 2r - (-r) = 3r.
%  Consistency: (Delta_nu^TED)^4 = 81 r^4 = 9 * (p^TED)^2. CHECKED.

peak_splitting   = (nu_plus_series - nu_minus_series);
% peak splitting at the TED is already -3r
peak_split_shift = (peak_splitting - 3*r);
peak_split_shift = collect(peak_split_shift, epsilon)
```

peak_split_shift =

$$\left(\frac{Z_1\, r}{3} - \frac{p_1}{2\, r}\right)\varepsilon + \varepsilon^{3/2}\left(\frac{5\ \sqrt{2}\ \sqrt{3}\ (-Z_1)^{3/2}\, r}{324} - \frac{\sqrt{2}\ \sqrt{3}\ \sqrt{-Z_1}\ p_1}{18\, r}\right) + \frac{\sqrt{2}\ \sqrt{3}\ \sqrt{-Z_1}\ \sqrt{\varepsilon}\ r}{3}$$

```
% a_sqrt^TED is the coefficient of sqrt(eps) in peak_split_shift
a_sqrt_TED_coeff = sqrt(6) * sqrt(-Z_1) * r / 3
```

a_sqrt_TED_coeff =

$$\frac{\sqrt{6}\ \sqrt{-Z_1}\ r}{3}$$

```
% Expected magnitude: sqrt(6)*sqrt(abs(Z_1))*r/3
```

Now, we see that there is a sqrt component, but there is also a linear and e^(3/2) component as well.

The next step is to convert Z_1 into physically-relevant parameters.

Reminder: This is Z_1

```
dZ_deps_at_TED == Z_1
```

ans =

$$-\frac{q_1 - p_1\, r}{2\, r^3} = Z_1$$

```
%% ------------------------------------------------------------------
%  Step 9: Bridge to physical parameters
%  ------------------------------------------------------------------
%  Express the Viete result in terms of observable quantities:
%      - |p^TED|   (local curvature of the cubic at the TED)
%      - ||grad Disc^TED|| * |sin alpha|  (the directional derivative D_1)
%
```

```
%  Bridge via direct CAS computation of dDisc/deps at eps = 0:

Disc_pert = -4*p_pert^3 - 27*q_pert^2;
D_1_raw = simplify( subs(diff(Disc_pert, epsilon), epsilon, 0) )
```

D_1_raw = $108\, r^3\, (q_1 - p_1\, r)$

The term in parentheses should be familiar! We can compare that to Z_1

```
% Substitute physical-branch relation: (q_1 - r*p_1) = 2*r^3 * Z_1
q_1_in_Z1 = solve(dZ_deps_at_TED == Z_1, q_1)
```

q_1_in_Z1 = $p_1\, r - 2\, Z_1\, r^3$

```
D_1_in_Z1 = simplify( subs(D_1_raw, q_1, q_1_in_Z1) )
```

D_1_in_Z1 = $-216\, Z_1\, r^6$

```

% reported in Paper - in terms of p_TED
subs(D_1_in_Z1, r, r_is_p)
```

ans = $8\, Z_1\, \widetilde{p}^3$

```

% Solve for Z_1 in terms of D_1 (treating D_1 as a single observable):
syms D_1 positive
Z_1_from_D1 = solve(D_1_in_Z1 == D_1, Z_1)
```

Z_1_from_D1 =

$$-\frac{D_1}{216\, r^6}$$

```

% Substitute into a_sqrt_TED_coeff:
a_sqrt_intermediate = simplify( subs(a_sqrt_TED_coeff, Z_1, Z_1_from_D1) )
```

a_sqrt_intermediate =

$$\frac{\sqrt{D_1}}{18\, r^2}$$

```
% Expected: sqrt(D_1) / (18 r^2)

% Final substitution using Vieta identity: |p^TED| = 3 r^2
syms p_TED_abs positive
a_sqrt_final = simplify( subs(a_sqrt_intermediate, r^2, p_TED_abs/3) )
```

a_sqrt_final =

$$\frac{\sqrt{D_1}}{6\, p_{\mathrm{TED,abs}}}$$

Alternative Drive/Readout Configurations (Appendix C.C)

The fact that the cubic equation that encodes all of the extrema is surprising, and we show here that it is unique to driving the cavity, reading out the yig (or, driving the yig, readout the cavity).

First, we show that Drive Cavity, Read YIG = Drive YIG, Read Cavity

Drive YIG - B = [0;1] // Read Cavity - C = [1 0]

```
beta_ss_dy_rc = simplify(abs([1 0] * inv(1j * f_tilde_d * eye(2) + A_tilde) * [0;1])^2, 'Steps'
simplify(beta_ss_dy_rc == beta_ss, 'Steps', 50)
```

```
ans = symtrue
```

To get intuition about the other configurations, note that the transmission is essentialy:

$\beta_{ss} \propto |\mathrm{CGB}|^2$ (we don't care about the drive strength u)

```
green
```

```
green =
```

$$\begin{pmatrix} -\frac{2\left(2\,\tilde{\Delta}_f - 2\,\tilde{\Delta}_\kappa\,\mathrm{i} - 2\,\tilde{f}_c + 2\,\tilde{f}_d + \tilde{\kappa}_c\,\mathrm{i}\right)}{\sigma_1} & -\frac{4}{\sigma_1} \\ -\frac{4\,\mathrm{e}^{\phi\,\mathrm{i}}}{\sigma_1} & -\frac{2\left(2\,\tilde{f}_d - 2\,\tilde{f}_c + \tilde{\kappa}_c\,\mathrm{i}\right)}{\sigma_1} \end{pmatrix}$$

where

$$\sigma_1 = 4\,\mathrm{e}^{\phi\,\mathrm{i}}\,\mathrm{i} + 4\,\tilde{\Delta}_f\,\tilde{f}_c\,\mathrm{i} - 4\,\tilde{\Delta}_f\,\tilde{f}_d\,\mathrm{i} + 4\,\tilde{\Delta}_\kappa\,\tilde{f}_c - 4\,\tilde{\Delta}_\kappa\,\tilde{f}_d + 2\,\tilde{\Delta}_f\,\tilde{\kappa}_c - 2\,\tilde{\Delta}_\kappa\,\tilde{\kappa}_c\,\mathrm{i} + 8\,\tilde{f}_c\,\tilde{f}_d\,\mathrm{i} - 4\,\tilde{f}_c\,\tilde{\kappa}_c + 4$$

When taking magnitudes, $G_{21} = G_{12}$, but the following also holds:

$G_{21} \neq G_{11}$
$G_{11} \neq G_{22}$

Thus, there is a fundamental difference between "cross-configurations" (drive one, readout the other) vs driving and reading the same port

For simplicity, we assume that there is no "mixed" driving or readout, such that B and C are both made up of 1 and 0.

Under this assumption, the transmission is equal to $|G_{\mathrm{DR}}|^2$, where D = the index of the mode being driven (1 for cavity, 2 for yig), and R = the index of the mode being readout.

```
% Cross config
cross_beta_ss = simplify(abs(green(2,1))^2, 'Steps', 100)
```

cross_beta_ss =

$$\frac{16}{\left(-4\,\tilde{f}_c^{\,2}+8\,\tilde{f}_c\,\tilde{f}_d+4\,\tilde{\Delta}_f\,\tilde{f}_c-4\,\tilde{f}_d^{\,2}-4\,\tilde{\Delta}_f\,\tilde{f}_d+\tilde{\kappa}_c^{\,2}-2\,\tilde{\Delta}_\kappa\,\tilde{\kappa}_c+4\cos(\phi)\right)^2+\left(4\sin(\phi)-4\,\tilde{\Delta}_\kappa\,\tilde{f}_c+4\right.}$$

```
% Drive/readout Cavity
cav_only_beta_ss = simplify(abs(green(1,1))^2, 'Steps', 100)
```

cav_only_beta_ss =

$$\frac{4\left(\left(2\,\tilde{\Delta}_\kappa-\tilde{\kappa}_c\right)^2+\left(2\,\tilde{\Delta}_f-2\,\tilde{f}_c+2\,\tilde{f}_d\right)^2\right)}{\left(-4\,\tilde{f}_c^{\,2}+8\,\tilde{f}_c\,\tilde{f}_d+4\,\tilde{\Delta}_f\,\tilde{f}_c-4\,\tilde{f}_d^{\,2}-4\,\tilde{\Delta}_f\,\tilde{f}_d+\tilde{\kappa}_c^{\,2}-2\,\tilde{\Delta}_\kappa\,\tilde{\kappa}_c+4\cos(\phi)\right)^2+\left(4\sin(\phi)-4\,\tilde{\Delta}_\kappa\,\tilde{f}_c+4\right.}$$

```
% Drive/readout YIG
yig_only_beta_ss = simplify(abs(green(2,2))^2, 'Steps', 100)
```

yig_only_beta_ss =

$$\frac{16\,\tilde{f}_c^{\,2}-32\,\tilde{f}_c\,\tilde{f}_d+16\,\tilde{f}_d^{\,2}+4\,\tilde{\kappa}_c^{\,2}}{\left(-4\,\tilde{f}_c^{\,2}+8\,\tilde{f}_c\,\tilde{f}_d+4\,\tilde{\Delta}_f\,\tilde{f}_c-4\,\tilde{f}_d^{\,2}-4\,\tilde{\Delta}_f\,\tilde{f}_d+\tilde{\kappa}_c^{\,2}-2\,\tilde{\Delta}_\kappa\,\tilde{\kappa}_c+4\cos(\phi)\right)^2+\left(4\sin(\phi)-4\,\tilde{\Delta}_\kappa\,\tilde{f}_c+4\right.}$$

We see here that for the drive+readout YIG or drive+readout Cavity configurations, the numerators are also functions of $\tilde{f}_d$! This means that the extrema expression will be more complicated.

Now look at the extrema polynomials for the three

```
dss = diff(cross_beta_ss, f_tilde_d) == 0;
s = solve(dss, f_tilde_d, 'ReturnConditions', true)
```

```
s = struct with fields:
     f_tilde_d: [3×1 sym]
    parameters: [1×0 sym]
    conditions: [3×1 sym]
```

```
s.f_tilde_d
```

ans =

$$\begin{pmatrix}\mathrm{root}(\sigma_1, z, 1)\\ \mathrm{root}(\sigma_1, z, 2)\\ \mathrm{root}(\sigma_1, z, 3)\end{pmatrix}$$

where

$$\sigma_1 = z^3+\frac{z^2\left(12\,\tilde{\Delta}_f-24\,\tilde{f}_c\right)}{8}+\frac{z\left(-8\cos(\phi)-24\,\tilde{\Delta}_f\,\tilde{f}_c-4\,\tilde{\Delta}_\kappa\,\tilde{\kappa}_c+4\,\tilde{\Delta}_f^{\,2}+4\,\tilde{\Delta}_\kappa^{\,2}+24\,\tilde{f}_c^{\,2}+2\,\tilde{\kappa}_c^{\,2}\right)}{8}.$$

We can see here that the extrema are based on a 3rd degree (cubic) polynomial, this is what we used in the text.

```
dss_cav = diff(cav_only_beta_ss, f_tilde_d) == 0;
s_cav = solve(dss_cav, f_tilde_d, 'ReturnConditions', true)
```

```
s_cav = struct with fields:
     f_tilde_d: x
    parameters: x
    conditions: 4*kappa_tilde_c^3*sin(phi) + 16*Delta_tilde_f^4*f_tilde_c + 16*Delta_tilde_kappa^4*f_tilde_c + f_til
```

```
s_cav.conditions
```

```
ans =
```

$$4\,\tilde{\kappa}_c^{\,3}\sin(\phi)+16\,\tilde{\Delta}_f^{\,4}\,\tilde{f}_c+16\,\tilde{\Delta}_\kappa^{\,4}\,\tilde{f}_c+\tilde{f}_c\,\tilde{\kappa}_c^{\,4}+x^2\left(-64\,\tilde{\Delta}_f^{\,3}+288\,\tilde{\Delta}_f^{\,2}\,\tilde{f}_c-64\,\tilde{\Delta}_f\,\tilde{\Delta}_\kappa^{\,2}+64\,\tilde{\Delta}_f\,\tilde{\Delta}_\kappa\,\tilde{\kappa}_c-\right.$$

where

$$\sigma_1 = 32\,\tilde{f}_c\,\tilde{\kappa}_c\sin(\phi)$$

$$\sigma_2 = 16\sin(\phi)^2$$

$$\sigma_3 = 16\,\tilde{\Delta}_f\,\tilde{\kappa}_c\sin(\phi)$$

$$\sigma_4 = 32\,\tilde{\Delta}_\kappa\,\tilde{f}_c\sin(\phi)$$

$$\sigma_5 = 16\cos(\phi)^2$$

$$\sigma_6 = 32\,\tilde{\Delta}_f\,\tilde{f}_c\cos(\phi)$$

For the drive/readout cavity, the conditions are implicitly defined as a **5th degree** (quintic) polynomial.

```
dss_yig = diff(yig_only_beta_ss, f_tilde_d) == 0;
s_yig = solve(dss_yig, f_tilde_d, 'ReturnConditions', true)
```

```
s_yig = struct with fields:
     f_tilde_d: x
    parameters: x
    conditions: 4*kappa_tilde_c^3*sin(phi) + f_tilde_c*kappa_tilde_c^4 + x*(16*cos(phi)^2 + 16*sin(phi)^2 + 64*Delta
```

Same, 5th degree polynomial.